浙江大学《BS 体系软件设计》课程大程

企业审批系统

设计文档

项目	内容
课程名称	《BS 体系软件设计》
实验项目	企业审批系统
学生姓名	李宇航
学　　号	3240105750
学　　院	人工智能学院
系	————————
专　　业	————————
指导教师	胡晓军
提交日期	2026 年 11 月 16 日前

目　录

1 项目环境....4
1.1 后端....4
1.2 前端....4
2 项目结构....6
2.1 后端....6
2.2 前端....7
3 数据库设计....8
3.1 E-R 图....8
3.2 表清单....10
3.3 删除策略....12
4 功能设计....13
4.1 后端....13
4.1.1 Model：数据库表定义....13
4.1.2 Repository：数据库操作....14
4.1.3 Config：配置信息....16
4.1.4 DTO：数据传输对象....17
4.1.5 Response：封装响应数据....19
4.1.6 Service：核心业务逻辑....21
4.1.7 Controller：接收前端请求....25
4.1.8 Main....28
4.2 前端....29
4.2.1 Lib：库/工具函数....29
4.2.2 Composables：组合式函数....31
4.2.3 Stores：全局状态....32
4.2.4 Components：组件....33
4.2.5 Views 与 Router：页面和路由....35
5 Q & A....38
Q1：附件和票据是如何存储的？....38
Q2：用户注册、登录时的信息是如何验证的？密码怎么存？....40
Q3：权限是如何控制的？为什么系统管理员不能审批？....40
Q4：申请人、部门、金额这些字段如何防止被前端篡改？....41
Q5：采购申请的报价规则如何校验？....41
Q6：差旅时长如何计算？差旅单如何被报销关联？....43
Q7：报销金额和票据如何校验？....44
Q8：管理员如何自定义审批流程？....45
Q9：审批流程是如何流转的？审批人如何确定？....49
Q10：撤回和审批同时发生怎么办？重复点击会不会重复流转？....52
Q11：驳回后如何重新提交？审批人离职怎么办？....55

Q12：审批日志和管理变更如何留痕、防止篡改？ 56
Q13：站内通知和实时提醒是如何实现的？ 57
Q14：统计数据的口径是什么？ 59
Q15：移动端如何适配？ 60
Q16：系统如何部署和初始化？ 61
Q17：增强功能是如何实现的？ 63

1 项目环境

1. 数据库：PostgreSQL 16
2. 文件存储：MinIO（S3 兼容对象存储，私有桶，存放报价单、票据等附件）
3. 邮件：SMTP（演示环境使用 Mailpit 收件）
4. 部署：Docker Compose + Nginx

选 PostgreSQL 而不是 MySQL，是因为设计里用到了它的几项能力：JSONB 存表单配置和流程快照，部分唯一索引做“同一报销单内票据不重复”“每个节点只有一个待办”这类约束，触发器让审批日志只能追加。附件放 MinIO 而不是数据库，下载时由后端鉴权后转发，浏览器拿不到直链。

1.1 后端

1. 核心框架：Maven 3.9 + Java 17 + Spring Boot 3.3
2. 安全：Spring Security 6 + JWT（`spring-boot-starter-oauth2-resource-server`），密码使用 BCrypt
3. 数据持久化：MyBatis-Plus 3.5 + PostgreSQL JDBC Driver
4. 数据库迁移：Flyway 10
5. 文件存储：MinIO Java SDK 8.5；文件类型识别：Apache Tika；PDF 检查：PDFBox
6. 其他：Spring Mail（邮件）、`SseEmitter`（实时推送）、Bucket4j（限流）、Caffeine（权限缓存）

持久层用 MyBatis-Plus 而不是 JPA：审批流转依赖大量“带条件的更新”（`WHERE status = ? AND version = ?`），影响行数决定操作是否成功，这类 SQL 写在 Mapper 里一目了然；JPA 的脏检查会把这一层藏起来。

1.2 前端

Web 端：

1. 核心框架：Vue 3.4 + TypeScript 5 + Vite 5
2. UI 组件库：Element Plus 2
3. 状态管理：Pinia 2；路由：Vue Router 4
4. 图表：ECharts 5
5. 网络请求：axios

移动端：

1. 核心框架：uni-app（Vue 3 + TypeScript），编译为 H5 和微信小程序

2. UI 组件：uni-ui
3. 图表：uCharts

移动端与 Web 端共用接口定义、类型和金额工具函数，页面单独实现，表格类内容在手机上改为卡片和分步表单。

2 项目结构

enterprise_approval/ Text

```
enterprise_approval
│  .gitignore
│  docker-compose.yml       # docker 配置文件
│  README.md
│  template.env             # .env 模板
│
├─backend                   # 后端
├─frontend                  # Web 前端
├─mobile                    # 移动端（uni-app）
├─nginx                     # Nginx 配置与镜像构建
└─docs
      design_document.pdf   # 设计文档
      user_guide.pdf        # 使用手册
```

2.1 后端

backend/ Text

```
backend
│  .gitignore
│  Dockerfile                    # docker 配置文件
│  pom.xml                       # Maven 配置文件
│
└─src/main
   ├─java/edu/zju/eas
   │  │  EasApplication.java      # 程序入口
   │  │
   │  ├─config         # 配置：安全、跨域、MyBatis-Plus、MinIO
   │  ├─controller     # 控制器，接收前端请求
   │  ├─dto            # 数据传输对象（请求体）
   │  ├─model          # 实体类，对应数据库表
   │  ├─repository     # Mapper 接口，定义数据库操作
   │  ├─response       # 统一响应格式与错误码
   │  └─service        # 服务层，处理核心业务逻辑
   │     ├─flow        # 流程引擎：审批人解析、状态流转
   │     ├─notify      # 站内通知、SSE 推送、邮件投递
   │     └─job         # 定时任务
   │
   └─resources
         application.yml         # 配置模板，敏感项取自环境变量
         application-demo.yml    # 演示环境配置
         db/migration            # Flyway 建表与基础数据脚本
         db/demo                 # 演示账号与示例申请
```

2.2 前端

frontend/ Text

```
frontend
│  .gitignore
│  Dockerfile              # docker 配置文件
│  index.html
│  package.json            # Node.js 配置文件
│  tsconfig.json
│  vite.config.ts          # Vite 配置，开发时代理 /api
│  template.env.local      # .env.local 模板
│
├─public                   # 公共资源
└─src
    │  App.vue             # 根组件
    │  main.ts             # 入口：挂载 Pinia、Router、Element Plus
    │
    ├─lib                  # API 封装、类型定义、工具函数
    ├─composables          # 组合式函数（相当于 Hooks）
    ├─stores               # Pinia 全局状态（相当于 Context）
    ├─components           # 组件
    ├─views                # 页面
    │  └─admin             # 管理端页面
    └─router               # 路由与守卫
```

mobile/ Text

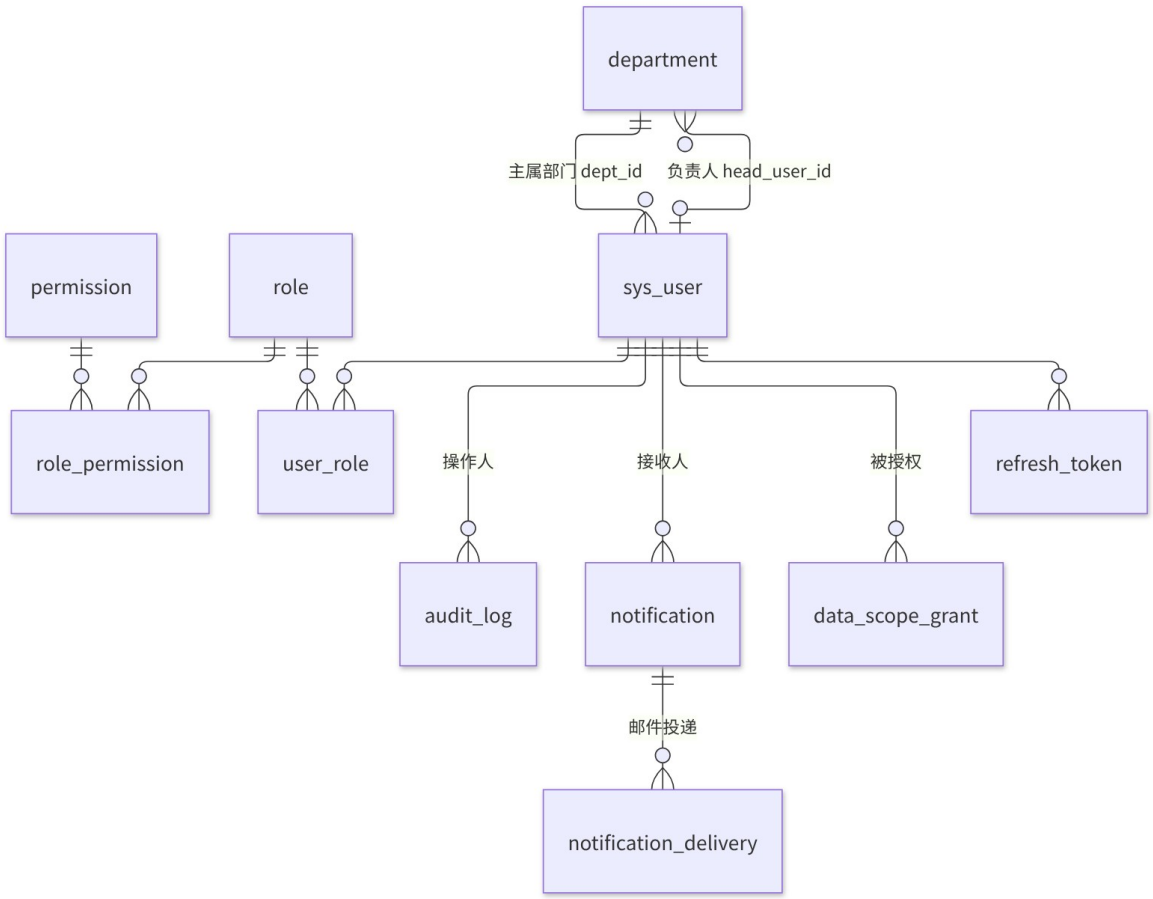
```
mobile
│  manifest.json          # 应用配置（H5、微信小程序）
│  pages.json             # 页面路由与 tabBar
│  package.json
│
├─pages                   # 页面
├─components              # 组件
├─lib                     # 与 Web 端共用的 api / types / money
└─static                  # 图标等静态资源
```

3 数据库设计

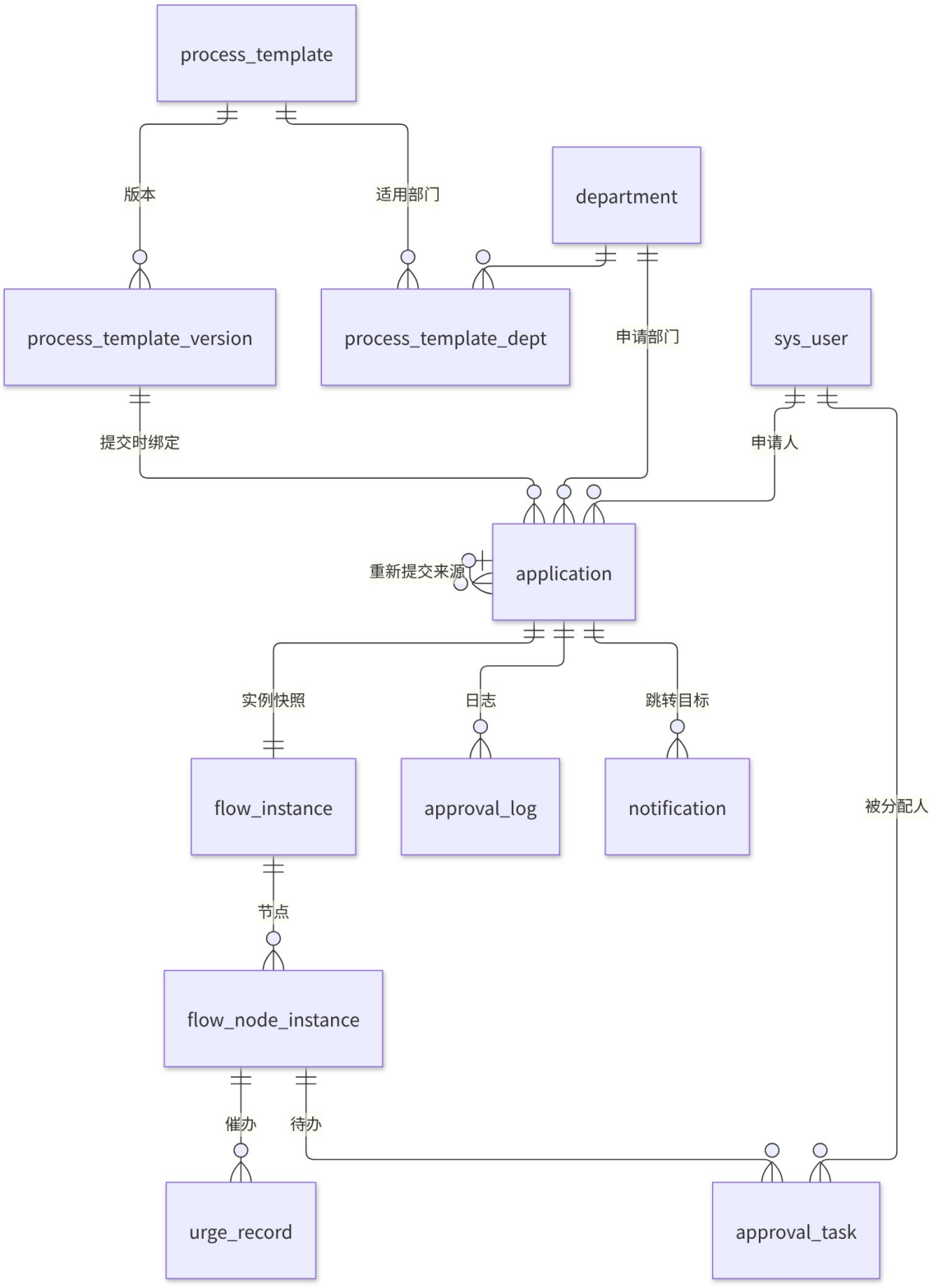
数据库共 34 张表，按用途分为三组：组织与权限、模板与流程、业务明细与附件。全部表结构由 Flyway 脚本创建，不手工改库。

3.1 E-R 图

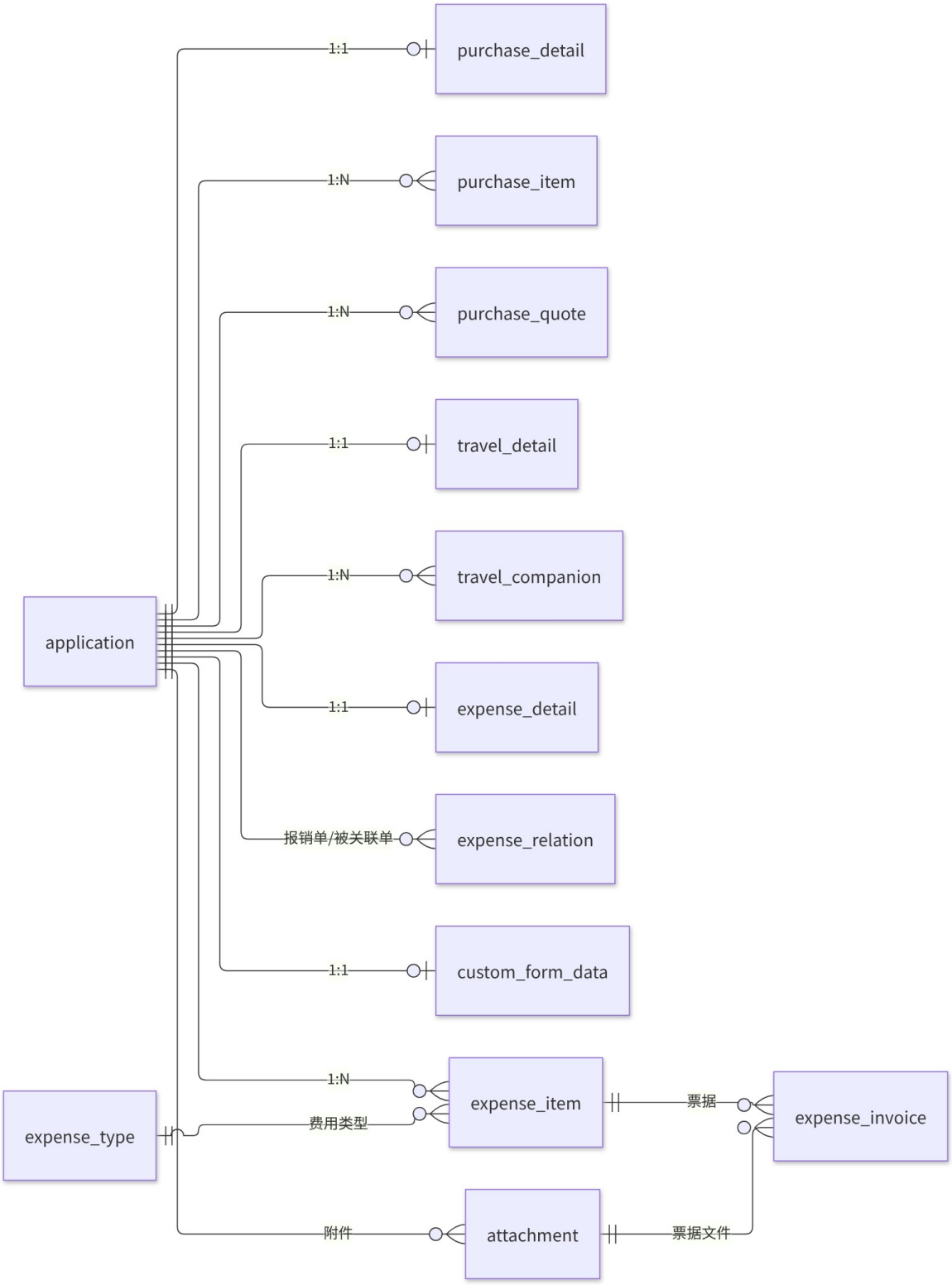
组织与权限：



模板、申请与流程：


process_template
版本
适用部门
department
process_template_version
process_template_dept
申请部门
sys_user
提交时绑定
申请人
重新提交来源
application
实例快照
日志
跳转目标
flow_instance
approval_log
notification
被分配人
节点
flow_node_instance
催办
待办
urge_record
approval_task

业务明细与附件：



3.2 表清单

列类型约定：主键 `BIGINT GENERATED ALWAYS AS IDENTITY`；时间 `TIMESTAMPTZ`；金额 `BIGINT`，单位为分，列名以 `_cents` 结尾；枚举用 `VARCHAR + CHECK`；半结构化配置用 `JSONB`。

表	说明	关键字段与约束
department	部门	code UK；name 在未删除范围内唯一；head_user_id → sys_user；deleted_at 软删除
sys_user	用户	lower(username)、lower(email) 唯一；password_hash（BCrypt）；status ∈ PENDING / ACTIVE / DISABLED / REJECTED；token_version 用于令牌失效；failed_login_count、locked_until
role / permission / role_permission	角色、权限点及关联	5 个内置角色；role_permission 随角色级联删除
user_role	用户角色	UK(user_id, role_id)；is_primary_handler 表示该角色的主办人，部分唯一索引保证每个角色至多一名
data_scope_grant	数据范围授权	permission_code、scope_type ∈ DEPT / ALL、有效期、原因；回收只写 revoked_at
refresh_token	刷新令牌	只存 SHA-256；family_id 用于重用检测
process_template	流程模板	code UK；category ∈ BUILTIN / CUSTOM；enabled；current_version_id
process_template_dept	模板适用部门	PK(template_id, dept_id)
process_template_version	模板版本	UK(template_id, version_no)；status ∈ DRAFT / PUBLISHED / ARCHIVED；form_schema、node_config 为 JSONB；每个模板至多一个 DRAFT、一个 PUBLISHED（部分唯一索引）；触发器禁止修改已发布版本
application	申请主表	app_no UK；type；status；applicant_id、dept_id（提交时固化）；amount_cents；current_node_seq；source_application_id（复制来源）；version（乐观锁）；deleted_at 只允许草稿
purchase_detail / purchase_item / purchase_quote	采购	报价 UK(application_id, lower(supplier_name))；线上报价必须有链接（CHECK）；单一来源必须有说明（CHECK）
travel_detail / travel_companion	差旅	CHECK(end_time ≥ start_time)；natural_days、duration_hours 由服务端写入；同行人为系统用户或文本姓名二选一
expense_type	费用类型	被引用后只能停用
expense_detail / expense_item	报销单与费用明细	total_amount_cents；无票明细必须有原因（CHECK）；UK(application_id, seq)
expense_invoice	票据	UK(application_id, file_sha256)；部分唯一 (application_id, invoice_no_norm) 防止同单重复票据
expense_relation	报销关联申请	PK(报销单, 被关联单)
custom_form_data	自定义表单值	form_values JSONB
flow_instance	流程实例	application_id UK；snapshot JSONB
flow_node_instance	节点实例	UK(flow_instance_id, seq)；status；assignee_id；部分唯一索引保证同一实例只有一个 ACTIVE 节点
approval_task	待办	status ∈ PENDING / APPROVED / REJECTED / CANCELED / TRANSFERRED；每个节点同一审批人至多一个 PENDING
approval_log	审批日志	操作人、动作、意见、前后状态、模板版本号；只追加

表	说明	关键字段与约束
audit_log	管理审计日志	动作、目标、before_data / after_data、原因、IP；只追加
attachment	附件	storage_key UK；size_bytes ≤ 10 MB（CHECK）；sha256；status ∈ TEMP / BOUND / DELETED
notification	站内通知	UK(recipient_id, dedup_key) 去重；is_read；link_path
notification_delivery	邮件投递	status、attempts、next_retry_at
idempotency_record	幂等记录	UK(user_id, idem_key)；request_hash；保存首次响应
urge_record	催办记录【增强】	UK(node_instance_id, time_bucket)，4 小时一档
sys_config	系统配置	兜底审批人、上传上限等

3.3 删除策略

外键默认 `ON DELETE RESTRICT`，只有随主单生灭的明细使用 `CASCADE`。

实体	策略
用户	不删除，改为 DISABLED；申请、日志、待办都引用用户
部门	软删除；仍有用户、申请、模板引用时拒绝
流程模板 / 版本	被申请引用后只能停用；已发布、已归档版本永久保留；DRAFT 版本可删
申请	只有草稿可软删除；定时任务物理清理 30 天前的软删草稿，明细随之 CASCADE
采购明细、报价、费用明细、票据、同行人、表单值	随申请 CASCADE
流程实例、节点、待办	永不删除
审批日志、审计日志	禁止 UPDATE / DELETE：触发器拒绝，运行账号也没有这两项权限
附件	TEMP 超过 24 小时未引用则删除；提交后的附件永久保留
授权	回收写 revoked_at，不删除

4 功能设计

4.1 后端

4.1.1 Model：数据库表定义

每张表对应一个实体类，按模块分包：

1. org：Department、SysUser、Role、Permission、UserRole、DataScopeGrant、RefreshToken
2. template：ProcessTemplate、ProcessTemplateVersion
3. app：Application，以及 PurchaseDetail、PurchaseItem、PurchaseQuote、TravelDetail、TravelCcompanion、ExpenseDetail、ExpenseItem、ExpenseInvoice、ExpenseType、CustomFormData
4. flow：FlowInstance、FlowNodeInstance、ApprovalTask、ApprovalLog
5. common：Attachment、Notification、NotificationDelivery、AuditLog、IdempotencyRecord、SysConfig

表结构本身由 Flyway 脚本创建（V1__init_schema.sql 起），实体类只负责映射，不用它反向建表。这样约束、部分唯一索引和触发器都写在 SQL 里，一处可查。

使用 MyBatis-Plus 注解映射表，以 Application 为例。

指定表名与主键：

Application.java

```java
@TableName(value = "application", autoResultMap = true) // 指定表名
public class Application {
    @TableId(type = IdType.AUTO)  // 主键由数据库 IDENTITY 生成
    private Long id;
    private String appNo;         // 驼峰自动映射到 app_no
}
```

申请人、部门、状态、金额这些字段只在服务层赋值，接口的请求体里没有对应字段：

Application.java Java

```java
public class Application {
    private String type;          // PURCHASE / TRAVEL / EXPENSE / CUSTOM
    private String title;
    private Long applicantId;     // 取自当前登录用户
    private Long deptId;          // 提交时固化为申请人主属部门
    private String status;        // DRAFT / IN_APPROVAL / APPROVED ...
    private Long amountCents;     // 统计金额，单位：分
    private Integer currentNodeSeq;
    private Long sourceApplicationId; // 复制重提的来源单
    private OffsetDateTime submittedAt;
    private OffsetDateTime finishedAt;
    private OffsetDateTime deletedAt; // 仅草稿可软删除
}
```

乐观锁字段：

Application.java Java

```java
public class Application {
    @Version            // 乐观锁版本号，所有状态变更都带 version 条件
    private Integer version;
}
```

JSONB 列通过类型处理器映射成对象，以模板版本为例：

ProcessTemplateVersion.java Java

```java
@TableName(value = "process_template_version", autoResultMap = true)
public class ProcessTemplateVersion {
    @TableId(type = IdType.AUTO)
    private Long id;
    private Long templateId;
    private Integer versionNo;
    private String status;        // DRAFT / PUBLISHED / ARCHIVED

    @TableField(typeHandler = JsonbTypeHandler.class)
    private FormSchema formSchema; // 表单字段定义

    @TableField(typeHandler = JsonbTypeHandler.class)
    private NodeConfig nodeConfig; // 审批节点定义
}
```

`JsonbTypeHandler` 继承 `AbstractJsonTypeHandler`，写入时把 JDBC 参数类型设为 `jsonb`，读取时用 Jackson 反序列化。

4.1.2 Repository：数据库操作

每个实体有一个 Mapper 接口，继承 `BaseMapper` 获得单表增删改查：

1. `ApplicationMapper`：申请主表
    1. 按申请人、状态、时间分页查询
    2. 按数据范围（本人负责部门、授权范围）分页查询
    3. 状态流转的条件更新
    4. ……
2. `ApprovalTaskMapper`：待办与已办查询、完成任务、取消任务
3. `FlowNodeInstanceMapper`：节点激活与结束
4. `ExpenseInvoiceMapper`：同单 / 跨单票据查询
5. `NotificationMapper`：未读数、标记已读、去重插入
6. `StatsMapper`：统计聚合查询
7. `SysUserMapper`、`DepartmentMapper`、`ProcessTemplateMapper` 等

以 `ApplicationMapper` 为例：

ApplicationMapper.java — Java

```java
@Mapper  // 由 @MapperScan 统一扫描
public interface ApplicationMapper extends BaseMapper<Application> {}
```

简单查询用 `LambdaQueryWrapper` 构造，字段名由方法引用得到，不写字符串：

ApplicationService.java — Java

```java
List<Application> drafts = applicationMapper.selectList(
    Wrappers.<Application>lambdaQuery()
        .eq(Application::getApplicantId, uid)
        .eq(Application::getStatus, "DRAFT")
        .isNull(Application::getDeletedAt)
        .orderByDesc(Application::getUpdatedAt));
```

状态流转用 `@Update` 直接写 SQL，返回影响行数。影响 0 行说明状态或版本已被别人改过，服务层据此回滚：

ApplicationMapper.java — Java

```java
public interface ApplicationMapper extends BaseMapper<Application> {
    @Update("""
        UPDATE application
           SET status = 'WITHDRAWN', version = version + 1,
               finished_at = now(), updated_at = now()
         WHERE id = #{id} AND applicant_id = #{uid}
           AND status = 'IN_APPROVAL' AND version = #{version}
        """)
    int withdraw(@Param("id") Long id, @Param("uid") Long uid,
                 @Param("version") int version);
}
```

带数据范围的列表查询写在 XML 中，范围条件直接进 SQL，分页的 `total` 不会把无权查看的记录算进去：

ApplicationMapper.xml SQL

```sql
-- scope=dept：只返回本人负责部门的非草稿申请
SELECT a.*
  FROM application a
 WHERE a.deleted_at IS NULL
   AND a.status <> 'DRAFT'
   AND a.dept_id IN (SELECT d.id FROM department d
                     WHERE d.head_user_id = #{uid}
                       AND d.deleted_at IS NULL)
   AND (#{type}   IS NULL OR a.type = #{type})
   AND (#{status} IS NULL OR a.status = #{status})
   AND (#{from}   IS NULL OR a.submitted_at >= #{from})
   AND (#{to}     IS NULL OR a.submitted_at <  #{to})
 ORDER BY a.submitted_at DESC
```

所有参数用 `#{}` 绑定，项目中禁止 `${}`；排序字段走白名单。

4.1.3 Config：配置信息

1. `SecurityConfig`：配置安全策略
    1. 无状态会话，关闭表单登录与 Session
    2. 公开接口：注册、登录、刷新令牌、部门列表、用户名 / 邮箱可用性检查；其余接口需登录
    3. `/api/v1/admin/**` 需要管理类权限点
    4. 定义密码编码器 `DelegatingPasswordEncoder`，默认 BCrypt（cost = 10）
    5. 注册 JWT 校验过滤器
2. `JwtConfig`：HS256 密钥来自环境变量 `JWT_SECRET`（≥ 32 字节）；访问令牌 30 分钟，刷新令牌 7 天
3. `CorsConfig`：开发时允许 `http://localhost:5173`（Vite）与 H5 调试地址；部署后前后端经 Nginx 同源，不再需要跨域
4. `MybatisPlusConfig`：注册分页插件与乐观锁插件
5. `MinioConfig`：创建 `MinioClient`，桶名 `eas-files`
6. `AsyncConfig`：开启 `@EnableScheduling`，配置定时任务线程池
7. `JacksonConfig`：时间按 ISO 8601 带偏移输出；请求中出现 DTO 未定义的字段时丢弃并记录 WARN

以 SecurityConfig 为例：

SecurityConfig.java Java

```java
@Configuration
@EnableMethodSecurity  // 开启 @PreAuthorize
public class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http,
                              JwtAuthFilter jwtFilter) throws Exception {
        http.csrf(c -> c.disable())   // 认证走 Authorization 头，不用 Cookie
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/v1/auth/register",
                                 "/api/v1/auth/login",
                                 "/api/v1/auth/refresh",
                                 "/api/v1/auth/availability",
                                 "/api/v1/public/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("SYS_ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        // 存储格式 {bcrypt}$2a$10$...，以后换算法不用迁移旧数据
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }
}
```

JwtAuthFilter 解析 Authorization: Bearer <token>，校验签名和过期时间后，再比对令牌中的 tv 与数据库中的 sys_user.token_version，并要求用户状态为 ACTIVE。用户被停用、角色变更或修改密码时 token_version + 1，旧令牌立即失效。用户的角色、权限点、负责部门和有效授权用 Caffeine 缓存 30 秒，管理员修改时主动清除。

CSRF 关闭的依据：API 认证依赖 Authorization 头，浏览器跨站请求不会自动带上它。唯一用 Cookie 的是 Web 端刷新令牌，该 Cookie 设置 HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth，刷新接口另外校验 Origin。

4.1.4 DTO：数据传输对象

DTO 用于接收前端请求体，只包含允许用户填写的字段。申请人、部门、状态、合计金额、审批人、角色都不在 DTO 中，客户端即使在 JSON 里带上也会被丢弃。

1. RegisterRequest：注册请求
2. LoginRequest：登录请求

3. `PurchaseSaveRequest`、`TravelSaveRequest`、`ExpenseSaveRequest`、`CustomSaveRequest`：保存草稿请求，带 `version`
4. `ActionRequest`：提交 / 撤回请求，带 `appVersion`
5. `ApproveRequest`、`RejectRequest`：审批意见与驳回理由
6. `ApplicationQuery`：列表筛选条件（类型、状态、提交时间区间、关键词、分页）
7. `TemplateSaveRequest`、`PublishRequest`：模板编辑与发布
8. `GrantCreateRequest`、`RoleGrantRequest`、`TransferRequest`：管理操作，均带必填的 `reason`

使用 Bean Validation 注解检查格式，以 `RegisterRequest` 为例：

RegisterRequest.java

```java
public record RegisterRequest(
    @NotBlank @Pattern(regexp = "^[a-zA-Z][a-zA-Z0-9_]{3,31}$",
        message = "用户名为 4-32 位字母、数字或下划线，以字母开头")
    String username,

    @NotBlank @Size(max = 50) String realName,

    @NotBlank @Email @Size(max = 100) String email,

    @NotBlank @Pattern(regexp = "^(?=.*[A-Za-z])(?=.*\\d).{8,64}$",
        message = "密码 8-64 位，至少包含字母和数字")
    String password,

    @NotNull Long requestedDeptId  // 申请加入的部门，不能选角色
) {}
```

报销单的请求体：明细金额是整数分，合计金额不接收：

ExpenseSaveRequest.java Java

```java
public record ExpenseSaveRequest(
    @NotNull Integer version,           // 草稿版本，用于并发保存检查
    @Size(max = 100) String title,
    @Size(max = 1000) String description,
    List<Long> relatedApplicationIds,   // 关联的差旅 / 采购单
    @Size(max = 500) String independentReason,
    @Size(max = 50) List<@Valid Item> items
) {
    public record Item(
        String expenseTypeCode,
        LocalDate occurDate,
        @Size(max = 200) String purpose,
        @Positive @Max(10_000_000) Long amountCents,  // 单条 ≤ 10 万元
        boolean noInvoice,
        @Size(max = 300) String noInvoiceReason,
        @Size(max = 10) List<Invoice> invoices
    ) {}
    public record Invoice(Long attachmentId,
                          @Size(max = 64) String invoiceNo) {}
}
```

草稿阶段只做格式校验（类型、长度、范围），必填项和业务规则在提交时检查，否则用户没法保存填了一半的草稿。

4.1.5 Response：封装响应数据

1. `ApiResponse<T>`：统一响应格式，所有接口返回 `{code, message, data, traceId}`
2. `PageResponse<T>`：分页数据 `{records, total, page, size}`
3. `ErrorCode`：错误码枚举，码值前三位与 HTTP 状态一致
4. `FieldError`：字段级错误 `{field, rule, message}`，`field` 可定位到明细行，如 `items[2].amountCents`
5. `GlobalExceptionHandler`：把异常转换成 `ApiResponse`，不向前端返回堆栈

以 `ApiResponse` 为例：

ApiResponse.java

```java
public record ApiResponse<T>(
    int code,           // 0 表示成功，其余见 ErrorCode
    String message,
    T data,
    String traceId,     // 与日志中的 traceId 对应，便于排查
    List<FieldError> errors
) {
    public static <T> ApiResponse<T> ok(T data) {
        return new ApiResponse<>(0, "OK", data, TraceContext.id(), null);
    }

    public static ApiResponse<Void> fail(ErrorCode ec,
                                         List<FieldError> errors) {
        return new ApiResponse<>(ec.code(), ec.message(), null,
                                 TraceContext.id(), errors);
    }
}
```

主要错误码：

code	HTTP	含义
0	200	成功
40000	400	参数格式错误，`errors` 定位字段
40001	400	缺少 `Idempotency-Key` 等请求头
40101 / 40102	401	未登录 / 访问令牌过期（前端自动刷新后重放）
40103	401	用户名或密码错误
40104 / 40105	403	账号待审核 / 账号停用或锁定
40301 / 40302 / 40303	403	无功能权限 / 不是当前审批人 / 不能审批自己的申请
40401	404	资源不存在或无权查看（两者不区分，防止探测）
40901 / 40902	409	申请状态已变化 / 待办已被处理
40903 / 40904	409	相同请求处理中 / 幂等键被用于不同请求
40905 / 40906	409	草稿已在别处修改 / 模板已更新
41301 / 41501	413 / 415	文件过大 / 文件类型不允许
42201	422	业务规则校验失败，`errors[].rule` 为规则码
42901	429	请求过于频繁
50000	500	服务器错误，界面显示 traceId

校验失败的响应示例：

响应示例 JSON

```json
{
  "code": 42201,
  "message": "业务规则校验失败",
  "traceId": "8f2c1a9e5b7d4c21",
  "errors": [
    {"field": "items[2].amountCents",
     "rule": "EXP_ITEM_AMOUNT",
     "message": "金额必须大于 0"},
    {"field": "items[1].invoices",
     "rule": "EXP_INVOICE_REQUIRED",
     "message": "请上传票据或勾选无票并填写原因"}
  ]
}
```

4.1.6 Service：核心业务逻辑

1. `AuthService`：用户认证
    1. 用户注册（状态为待审核）
    2. 用户登录、失败计数与锁定
    3. 刷新令牌轮换、退出登录
    4. 修改密码
2. `UserAdminService`：注册审核、启用 / 停用、角色授予与回收、设置角色主办人、重置密码
3. `DepartmentService`：部门增改、软删除、设置负责人（同步授予 / 回收 `DEPT_HEAD` 角色）
4. `GrantService`：数据范围授权的创建、回收与判断 `covers(userId, permission, deptId)`
5. `AccessPolicy`：资源级权限判断，申请、附件、关联单的查看权都经过它
6. `ApplicationService`：申请的通用操作
    1. 新建、保存、删除草稿
    2. 复制重提
    3. 详情组装（业务数据、流程节点、审批日志、当前用户可执行的操作）
    4. 列表查询
7. `PurchaseService`、`TravelService`、`ExpenseService`、`CustomFormService`：各类申请的字段保存与提交校验
8. `TemplateService`：模板增改、启用 / 停用、草稿版本、发布
9. `TemplatePublishValidator`：发布前校验
10. `flow` 包：流程引擎

    1. `FlowEngine`：提交、批准、驳回、撤回、转办
    2. `ApproverResolver`：把节点规则解析为具体审批人
    3. `ConditionEvaluator`：节点条件求值
11. `FileService`：附件上传、类型检测、绑定、鉴权下载
12. `notify` 包：`NotificationService`（写站内通知）、`SseHub`（推送）、`MailDispatcher`（邮件投递）
13. `StatsService`：统计查询
14. `AuditService`：管理操作审计
15. `IdempotencyService`：幂等键登记与结果重放
16. `job` 包：附件清理、幂等记录清理、软删草稿清理、超时提醒

`@Service` 标注服务类，写操作方法加 `@Transactional`：

AuthService.java — Java

```java
@Service
@RequiredArgsConstructor  // 构造器注入
public class AuthService {}
```

用户注册：

AuthService.java — Java

```java
public class AuthService {
    @Transactional
    public Long register(RegisterRequest req) {
        // 唯一性检查；数据库 lower(username) 唯一索引兜底并发注册
        if (userMapper.existsUsername(req.username())) {
            throw new BizException(ErrorCode.USERNAME_OR_EMAIL_TAKEN);
        }
        if (userMapper.existsEmail(req.email())) {
            throw new BizException(ErrorCode.USERNAME_OR_EMAIL_TAKEN);
        }
        deptService.requireEnabled(req.requestedDeptId());

        SysUser u = new SysUser();
        u.setUsername(req.username());
        u.setRealName(req.realName());
        u.setEmail(req.email().toLowerCase());
        u.setPasswordHash(passwordEncoder.encode(req.password()));
        u.setRequestedDeptId(req.requestedDeptId());
        u.setStatus("PENDING");   // 待管理员审核，不授予任何角色
        userMapper.insert(u);
        return u.getId();
    }
}
```

用户登录：

AuthService.java

Java

```java
public class AuthService {
    @Transactional(noRollbackFor = BizException.class)
    public TokenPair login(LoginRequest req, ClientInfo client) {
        SysUser u = userMapper.findByUsernameOrEmail(req.account());
        if (u != null && u.isLocked(clock)) {
            throw new BizException(ErrorCode.ACCOUNT_DISABLED_OR_LOCKED);
        }
        if (u == null
                || !passwordEncoder.matches(req.password(),
                                            u.getPasswordHash())) {
            // 15 分钟内连续失败 5 次，锁定 15 分钟
            if (u != null) userMapper.recordFailure(u.getId());
            // 不区分账号不存在与密码错误
            throw new BizException(ErrorCode.BAD_CREDENTIALS);
        }
        if (!"ACTIVE".equals(u.getStatus())) {
            throw new BizException("PENDING".equals(u.getStatus())
                ? ErrorCode.ACCOUNT_PENDING
                : ErrorCode.ACCOUNT_DISABLED_OR_LOCKED);
        }
        userMapper.resetFailure(u.getId());
        return tokenService.issue(u, client);  // access + refresh
    }
}
```

`AccessPolicy` 判断能否查看一张申请：

AccessPolicy.java

Java

```java
public class AccessPolicy {
    public boolean canViewApplication(CurrentUser u, Application app) {
        if (app.getDeletedAt() != null) return false;
        // 本人申请（含草稿）
        if (app.getApplicantId().equals(u.id())) return true;
        // 他人草稿一律不可见
        if ("DRAFT".equals(app.getStatus())) return false;
        // 当前或历史上被分配过该申请的待办
        if (taskMapper.existsByAppAndAssignee(app.getId(), u.id())) {
            return true;
        }
        // 本人负责的部门
        if (u.has(APP_READ_DEPT)
                && deptMapper.isHead(app.getDeptId(), u.id())) {
            return true;
        }
        // 管理员授予的 DEPT / ALL 范围，且在有效期内
        if (grantService.covers(u.id(), APP_READ, app.getDeptId())) {
            if (u.isAdmin()) {
                auditService.logBizAccess(u, app); // 管理员看正文留痕
            }
            return true;
        }
        return false;
    }
}
```

无查看权时接口返回 404 而不是 403，不暴露“这张单存在”。`SYS_ADMIN` 角色只有管理类权限点，不能审批，默认也读不到业务正文。

流程引擎的批准操作。approve 在一个事务里完成：推进申请、完成任务、激活下一节点、写日志和通知。

FlowEngine.java Java

```java
public class FlowEngine {
    @Transactional
    public ActionResult approve(Long taskId, CurrentUser u,
                                ApproveRequest req) {
        ApprovalTask task = taskMapper.selectById(taskId);
        Application app = appMapper.selectById(task.getApplicationId());
        if (app.getApplicantId().equals(u.id())) {
            throw new BizException(ErrorCode.SELF_APPROVAL_FORBIDDEN);
        }
        FlowNodeInstance node = nodeMapper.selectById(task.getNodeId());
        FlowNodeInstance next = nodeMapper.nextEffective(node);
        boolean isLast = (next == null);

        // ① 先锁申请行：状态、版本、当前节点都对得上才更新
        int n = appMapper.advance(app.getId(), req.appVersion(),
            node.getSeq(), isLast ? null : next.getSeq(), isLast, u.id());
        if (n == 0) throw new StateChangedException();      // 40901

        // ② 再完成任务：只能是本人的 PENDING 任务
        if (taskMapper.complete(taskId, u.id(), "APPROVED",
                                req.comment()) == 0) {
            throw new TaskProcessedException();             // 40902
        }

        nodeMapper.finish(node.getId(), "APPROVED");
        ApprovalTask newTask = isLast ? null : activate(next, app);

        logService.append(app, node, task, u, "APPROVE", req.comment());
        notifyService.onApproved(app, node, newTask);  // 同事务写通知
        return ActionResult.of(appMapper.selectById(app.getId()));
    }
}
```

所有流转事务都先更新 application 行，再更新 approval_task 行。撤回从申请出发、审批从任务出发，加锁顺序统一后不会互相死锁。撤回的写法和并发处理见第 5 章 Q10。

4.1.7 Controller：接收前端请求

1. AuthController（/api/v1/auth）：注册、登录、刷新令牌、退出、可用性检查、SSE 票据
2. MeController（/api/v1/me）：个人信息、修改密码、邮件提醒开关
3. ApplicationController（/api/v1/applications）：
    1. 各类申请的新建与保存草稿
    2. 列表、详情、审批日志

3. 提交、撤回、删除草稿、复制重提
4. 审批路径预览、可关联申请
5. 催办【增强】、跨单重复票据提示【增强】

4. `TaskController`（`/api/v1/tasks`）：待办、已办、批准、驳回
5. `TemplateController`（`/api/v1/templates`）：可发起的模板、模板当前版本
6. `FileController`（`/api/v1/files`）：上传、预览、下载
7. `NotificationController`（`/api/v1/notifications`）：列表、未读数、标记已读、全部已读、SSE 连接、增量拉取
8. `StatsController`（`/api/v1/stats`）：个人、本部门、财务、企业统计
9. `admin` 包（`/api/v1/admin/**`）：用户与注册审核、部门、角色、授权、模板设计与发布、费用类型、系统配置、待办监控与转办、审计日志

主要接口：

方法	路径	权限	说明
POST	`/auth/register`	公开	注册，状态为待审核
POST	`/auth/login`	公开（限流）	登录，返回访问令牌
POST	`/auth/refresh`	刷新令牌	轮换刷新令牌
POST	`/applications/{type}`	`APP_CREATE`	新建草稿，`type` 为 purchase / travel / expense / custom
PUT	`/applications/{id}/{type}`	本人草稿	保存草稿，带 `version`
GET	`/applications?scope=`	登录，按范围	`scope` 取 mine / dept / granted；类型、状态、提交时间、关键词筛选，分页
GET	`/applications/{id}`	可查看	详情，含 `allowedActions`
POST	`/applications/{id}/submit`	本人草稿 + 幂等键	提交
POST	`/applications/{id}/withdraw`	本人 + 幂等键	撤回
DELETE	`/applications/{id}`	本人草稿	删除草稿
POST	`/applications/{id}/copy`	本人、已驳回 / 已撤回	复制为新草稿
GET	`/tasks/pending`、`/tasks/done`	`TASK_HANDLE`	待办、已办
POST	`/tasks/{taskId}/approve`	当前审批人 + 幂等键	批准
POST	`/tasks/{taskId}/reject`	当前审批人 + 幂等键	驳回，理由必填
POST	`/files`	登录（限流）	上传附件
GET	`/files/{id}/content`	可读附件	预览 / 下载
GET	`/notifications/unread-count`	登录	未读数

方法	路径	权限	说明
POST	/notifications/{id}/read	本人通知	标记已读
GET	/notifications/stream?ticket=	一次性票据	SSE 连接
GET	/stats/dept	STATS_DEPT	本部门统计
POST	/admin/templates/{id}/publish	ADMIN_TEMPLATE	发布模板版本
POST	/admin/tasks/{taskId}/transfer	ADMIN_TRANSFER	转办，需填原因

`@RestController` 标注控制器，`@RequestMapping` 指定基础路径：

AuthController.java Java

```java
@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {}
```

用户注册，调用 Service 层：

AuthController.java Java

```java
public class AuthController {
    @PostMapping("/register")
    public ApiResponse<Map<String, Long>> register(
            @Valid @RequestBody RegisterRequest req) {
        Long id = authService.register(req);
        return ApiResponse.ok(Map.of("userId", id));
    }
}
```

批准待办。`@PreAuthorize` 只检查功能权限，是否为当前审批人由 `FlowEngine` 在 SQL 条件里判断：

TaskController.java Java

```java
@RestController
@RequestMapping("/api/v1/tasks")
public class TaskController {
    @PostMapping("/{taskId}/approve")
    @PreAuthorize("hasAuthority('TASK_HANDLE')")
    @Idempotent  // 拦截器检查 Idempotency-Key，重复请求返回首次结果
    public ApiResponse<ActionResult> approve(
            @PathVariable Long taskId,
            @Valid @RequestBody ApproveRequest req,
            @AuthenticationPrincipal CurrentUser user) {
        return ApiResponse.ok(flowEngine.approve(taskId, user, req));
    }
}
```

请求与响应示例：

请求示例 HTTP

```
POST /api/v1/tasks/1203/approve
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Idempotency-Key: 6f1c9d1e-3b7a-4f7e-9a0d-2c5e8b4a7f10
Content-Type: application/json

{"appVersion": 5, "comment": "同意"}
```

响应示例 JSON

```
{
  "code": 0,
  "message": "OK",
  "data": {
    "applicationId": 305,
    "status": "IN_APPROVAL",
    "appVersion": 6,
    "currentNode": {"seq": 2, "name": "财务审核",
                    "assigneeName": "吴审"}
  },
  "traceId": "b41e7c0d92aa4f3e"
}
```

4.1.8 Main

启动 Spring Boot 服务，扫描 Mapper，开启定时任务：

EasApplication.java Java

```
@SpringBootApplication
@MapperScan("edu.zju.eas.repository")
@EnableScheduling
public class EasApplication {
    public static void main(String[] args) {
        // 统一使用上海时区计算自然日与统计日期
        TimeZone.setDefault(TimeZone.getTimeZone("Asia/Shanghai"));
        SpringApplication.run(EasApplication.class, args);
    }

    @Bean
    Clock clock() {  // 注入 Clock，测试时可替换为固定时间
        return Clock.system(ZoneId.of("Asia/Shanghai"));
    }
}
```

启动时 Flyway 先执行迁移脚本，迁移失败则应用不启动；`demo` profile 额外加载 `db/demo` 下的演示数据。

4.2 前端

4.2.1 Lib：库/工具函数

1. `http.ts`：axios 实例与拦截器
    1. 请求时带上访问令牌
    2. 收到 `40102` 时刷新令牌并重放原请求，并发请求只刷新一次
    3. 统一把 `ApiResponse` 拆成 `data` 或抛出带 `code`、`errors` 的异常
2. `api/*.ts`：按模块封装后端接口（`auth`、`applications`、`tasks`、`files`、`notifications`、`stats`、`admin`）
3. `types.ts`：前后端共享的数据结构（申请、明细、待办、通知、模板 Schema 等）
4. `money.ts`：元与分的转换，全部按字符串处理
5. `idem.ts`：生成幂等键

以 `http.ts` 为例：

src/lib/http.ts

```typescript
const http = axios.create({ baseURL: import.meta.env.VITE_API_BASE });

http.interceptors.request.use((config) => {
  const token = useAuthStore().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

let refreshing: Promise<void> | null = null;

http.interceptors.response.use(
  (res) => {
    const body = res.data as ApiResponse<unknown>;
    if (body.code !== 0) throw new ApiError(body);
    return body.data as any;
  },
  async (err) => {
    const body = err.response?.data as ApiResponse<unknown> | undefined;
    if (body?.code === 40102 && !err.config._retried) {
      // 多个请求同时过期时只刷新一次
      refreshing ??= useAuthStore().refresh()
        .finally(() => (refreshing = null));
      await refreshing;
      err.config._retried = true;
      return http(err.config);
    }
    throw body ? new ApiError(body) : err;
  },
);
```

状态流转类请求带上幂等键。键在用户打开确认框时生成，同一次操作的重试复用同一个键：

src/lib/api/tasks.ts

```typescript
export function approveTask(taskId: number, body: ApproveBody,
                            idemKey: string) {
  return http.post<ActionResult>(`/tasks/${taskId}/approve`, body, {
    headers: { 'Idempotency-Key': idemKey },
  });
}
```

金额转换不经过 `parseFloat`，避免 `0.1 + 0.2` 这类浮点误差：

src/lib/money.ts

```typescript
// "12.3" -> 1230, "12.30" -> 1230; 非法输入返回 null
export function yuanToCents(input: string): number | null {
  const m = /^(\d{1,6})(?:\.(\d{1,2}))?$/.exec(input.trim());
  if (!m) return null;
  const frac = (m[2] ?? '').padEnd(2, '0');
  return Number(m[1]) * 100 + Number(frac);
}

// 1230 -> "12.30"
export function centsToYuan(cents: number): string {
  const yuan = Math.trunc(cents / 100);
  const frac = String(cents % 100).padStart(2, '0');
  return `${yuan}.${frac}`;
}
```

页面上的合计只用于即时显示，以服务端重算的结果为准。

4.2.2 Composables：组合式函数

Vue 3 的组合式函数相当于 React 的 Hooks，把可复用的状态和逻辑抽出来：

1. `useListQuery.ts`：列表的筛选条件、分页与 URL 查询参数同步，刷新页面后筛选不丢
2. `useDraftForm.ts`：草稿表单的加载、保存、`version` 维护，有未保存修改时离开页面给出提示
3. `useAction.ts`：提交、撤回、批准、驳回等操作的二次确认、loading 状态、幂等键、409 时提示刷新
4. `useSse.ts`：SSE 连接、断线重连与降级轮询
5. `useBreakpoint.ts`：判断窗口是否为移动端宽度

以 `useSse.ts` 为例：

src/composables/useSse.ts TypeScript

```typescript
export function useSse(onMessage: (n: NoticeEvent) => void) {
  let es: EventSource | null = null;
  let failures = 0;
  let pollTimer: number | undefined;

  async function connect() {
    // EventSource 不能带请求头，先用访问令牌换一次性票据
    const { ticket } = await api.notifications.sseTicket();
    es = new EventSource(`/api/v1/notifications/stream?ticket=${ticket}`);
    es.addEventListener('notification', (e) => {
      failures = 0;
      onMessage(JSON.parse((e as MessageEvent).data));
    });
    es.onerror = () => {
      es?.close();
      // 连续失败 3 次后改为每 15 秒轮询未读数
      if (++failures >= 3) startPolling();
      else setTimeout(connect, 2000 * failures);
    };
  }

  function startPolling() {
    pollTimer = window.setInterval(async () => {
      useNoticeStore().setUnread(await api.notifications.unreadCount());
    }, 15_000);
  }

  onMounted(connect);
  onUnmounted(() => { es?.close(); clearInterval(pollTimer); });
}
```

4.2.3 Stores：全局状态

Pinia store 相当于 React 的 Context，被多个页面共享的状态放在这里，任何组件都可以直接读取和修改：

1. `auth.ts`：当前用户、访问令牌、角色与权限点、负责的部门
2. `notice.ts`：未读数、最近通知
3. `dict.ts`：部门、费用类型等字典数据，首次使用时加载

以 auth.ts 为例：

src/stores/auth.ts TypeScript

```typescript
export const useAuthStore = defineStore('auth', {
  state: () => ({
    accessToken: sessionStorage.getItem('at') as string | null,
    me: null as Me | null,
  }),
  getters: {
    isLoggedIn: (s) => !!s.accessToken,
    // 只决定菜单和按钮是否显示，真正的权限检查在后端
    can: (s) => (perm: string) => !!s.me?.permissions.includes(perm),
    isDeptHead: (s) => (s.me?.headDeptIds.length ?? 0) > 0,
  },
  actions: {
    async login(account: string, password: string) {
      const r = await api.auth.login({ account, password });
      this.setToken(r.accessToken);
      this.me = await api.me.get();
    },
    async refresh() {
      // Web 端刷新令牌在 HttpOnly Cookie 中，JS 读不到
      const r = await api.auth.refresh();
      this.setToken(r.accessToken);
    },
    setToken(t: string | null) {
      this.accessToken = t;
      t ? sessionStorage.setItem('at', t) : sessionStorage.removeItem('at');
    },
    async logout() {
      await api.auth.logout().catch(() => {});
      this.setToken(null);
      this.me = null;
    },
  },
});
```

访问令牌放在 sessionStorage 而不是 localStorage，关闭标签页即失效；刷新令牌不交给 JS。

4.2.4 Components：组件

1. StatusTag.vue：状态标签。草稿灰色、审批中蓝色（附当前节点与审批人）、已通过绿色、已驳回红色、已撤回深灰
2. AmountInput.vue：金额输入，内部以字符串保存，对外输出整数分
3. MoneyText.vue：金额显示，千分位、两位小数
4. ItemTable.vue：可增删行的明细表，用于采购明细与报销明细
5. QuoteTable.vue：报价比较表，最低价高亮，标出拟选供应商

6. `AttachmentUploader.vue`：附件上传，前端预检类型、大小与数量
7. `FlowTimeline.vue`：流程时间线，显示每个节点的审批人、状态、意见和时间，当前节点高亮
8. `ApprovePanel.vue`：批准 / 驳回面板，驳回理由必填
9. `FormRenderer.vue`：按模板 `form_schema` 动态渲染自定义表单
10. `TemplateDesigner/`：管理端流程设计器（字段库、表单画布、字段属性、节点列表、条件编辑器）
11. `StatsChart.vue`：ECharts 图表封装
12. `NoticeBell.vue`：顶部通知铃铛与未读角标

以报销明细表中的一行为例，使用 Element Plus 组件搭建：

src/components/ExpenseItemTable.vue

```vue
<template>
  <el-table :data="items" border>
    <el-table-column label="费用类型" width="130">
      <template #default="{ row }">
        <el-select v-model="row.expenseTypeCode" placeholder="请选择">
          <el-option v-for="t in dict.expenseTypes" :key="t.code"
                     :label="t.name" :value="t.code" />
        </el-select>
      </template>
    </el-table-column>
    <el-table-column label="金额（元）" width="140">
      <template #default="{ row }">
        <AmountInput v-model:cents="row.amountCents" />
      </template>
    </el-table-column>
    <el-table-column label="票据">
      <template #default="{ row }">
        <el-checkbox v-model="row.noInvoice">无票</el-checkbox>
        <el-input v-if="row.noInvoice" v-model="row.noInvoiceReason"
                  placeholder="无票原因（至少 10 字）" />
        <AttachmentUploader v-else v-model="row.invoices"
```

src/components/ExpenseItemTable.vue（续） Vue

```vue
                            :max-count="10" ref-type="EXPENSE_INVOICE" />
      </template>
    </el-table-column>
    <el-table-column width="70">
      <template #default="{ $index }">
        <el-button link type="danger"
                   @click="items.splice($index, 1)">删除</el-button>
      </template>
    </el-table-column>
  </el-table>
  <div class="total">合计：<MoneyText :cents="totalCents" /></div>
</template>

<script setup lang="ts">
const items = defineModel<ExpenseItem[]>({ required: true });
const dict = useDictStore();
// 整数分相加，不涉及浮点
const totalCents = computed(() =>
  items.value.reduce((s, i) => s + (i.amountCents ?? 0), 0));
</script>
```

所有用户输入（文件名、意见、表单值）都按文本插值渲染，项目禁用 `v-html`（ESLint 规则 `vue/no-v-html` 设为 error）。

4.2.5 Views 与 Router：页面和路由

1. `router/index.ts`：路由表与全局守卫。守卫只负责体验（未登录跳转登录页、无权限的菜单不显示），权限以后端为准
2. `App.vue`：根组件，包含顶部导航、侧边菜单和通知铃铛
3. `views/`：各页面，路由如下

路由	页面	说明
/login、/register	登录、注册	注册时实时检查用户名 / 邮箱是否可用，提示密码强度
/	工作台	待办数、我的申请状态、最近通知、快捷发起
/apply	发起申请	采购、差旅、报销及已启用的自定义模板
/apply/purchase	采购表单	申请信息、采购明细、报价比较、采购选择、附件
/apply/travel	差旅表单	选完起止时间后显示“自然日 N 天 / 实际 H 小时”
/apply/expense	报销表单	关联申请选择、费用明细、票据、即时合计
/apply/custom/:templateId	自定义表单	FormRenderer 渲染
/applications/:id/edit	编辑草稿	按类型复用上述表单
/applications/mine	我的申请	类型、状态、提交时间、关键词筛选；查看、编辑、删除、撤回、复制重提
/applications/:id	申请详情	业务信息、附件、流程时间线、审批日志；按 allowedActions 显示按钮

路由	页面	说明
/tasks/pending、/tasks/done	待办、已办	处理待办时打开详情与审批面板
/dept/applications	本部门申请	部门负责人可见
/stats	统计	个人 / 部门 / 财务 / 企业，按权限显示
/notifications	通知中心	全部 / 未读、标记已读、点击跳转
/profile	个人信息	修改密码、邮件提醒开关
/admin/users、/admin/registrations	用户管理、注册审核	审核、分配部门、授予角色
/admin/departments	部门管理	设置负责人
/admin/grants	授权管理	权限点、范围、有效期、原因
/admin/templates/:id/design	流程设计器	表单与节点配置、发布前校验、发布
/admin/tasks	待办监控与转办	只显示单据元数据
/admin/audit-logs	审计日志	前后值对比

路由表中用 `meta` 声明菜单需要的权限点：

src/router/index.ts — TypeScript

```ts
const routes: RouteRecordRaw[] = [
  { path: '/login', component: () => import('@/views/Login.vue'),
    meta: { public: true } },
  { path: '/', component: Layout, children: [
    { path: '', component: () => import('@/views/Workbench.vue') },
    { path: 'applications/:id', props: true,
      component: () => import('@/views/ApplicationDetail.vue') },
    { path: 'tasks/pending', meta: { perm: 'TASK_HANDLE' },
      component: () => import('@/views/TaskPending.vue') },
    { path: 'admin/templates/:id/design', props: true,
      meta: { perm: 'ADMIN_TEMPLATE' },
      component: () => import('@/views/admin/TemplateDesigner.vue') },
  ]},
];

router.beforeEach((to) => {
  const auth = useAuthStore();
  if (!to.meta.public && !auth.isLoggedIn) {
    return { path: '/login', query: { redirect: to.fullPath } };
  }
  if (to.meta.perm && !auth.can(to.meta.perm as string)) return '/403';
});
```

移动端（uni-app）页面在 `pages.json` 中声明，底部 tabBar 为工作台、发起、待办、消息、我的：

页面路径	功能	适配要点
pages/home/index	工作台	卡片布局
pages/apply/purchase	采购	分步填写：基本信息 → 明细 → 报价 → 选择 → 附件 → 确认；明细与报价为卡片列表
pages/apply/travel	差旅	日期时间选择器，实时显示天数

页面路径	功能	适配要点
pages/apply/expense	报销	每条明细卡片内可拍照、选相册或选 PDF；底部固定栏显示合计
pages/apply/custom	自定义表单	动态表单的移动版
pages/app/list、pages/app/detail	我的申请、详情	列表为卡片，详情时间线纵向排列，底部操作栏
pages/task/list、pages/task/handle	待办、审批	底部“批准 / 驳回”双按钮
pages/notice/list	消息	tabBar 角标显示未读数
pages/me/index、pages/stats/index	我的、统计	uCharts 图表

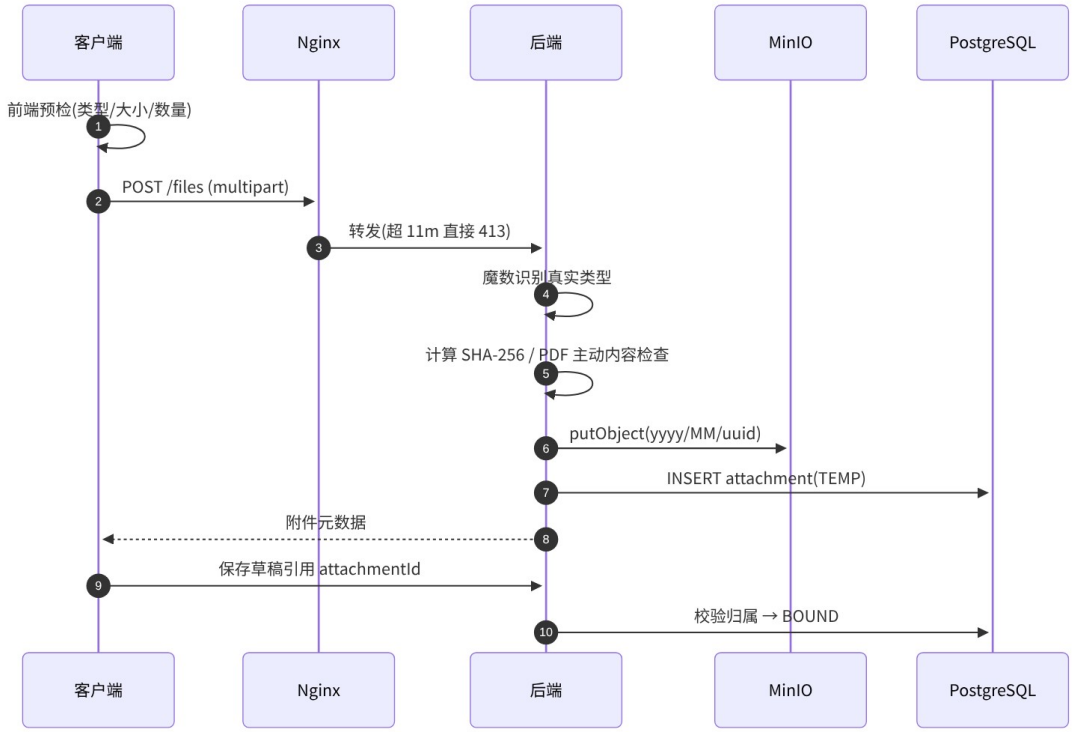
5 Q & A

Q1：附件和票据是如何存储的？

文件存放在 MinIO 私有桶 `eas-files` 中，数据库 `attachment` 表只记录元数据：原始文件名、上传人、上传时间、大小、服务端检测到的类型、SHA-256 和对象键。对象键格式为 `yyyy/MM/{uuid}`，与原文件名无关，文件名里的特殊字符不会影响存储路径。

浏览器拿不到 MinIO 地址。预览和下载都请求 `GET /files/{id}/content`，后端先调用 `AccessPolicy.canReadAttachment` 判断权限，再从 MinIO 读出文件流转发给浏览器，所以权限检查绕不过去。

上传流程：



文件类型以文件头魔数为准，不信任扩展名和浏览器给的 `Content-Type`：

FileService.java — Java

```java
public class FileService {
    private static final Set<String> ALLOWED =
        Set.of("image/jpeg", "image/png", "application/pdf");

    public Attachment upload(MultipartFile file, String refType,
                             CurrentUser u) throws IOException {
        if (file.getSize() > 10L * 1024 * 1024) {
            throw new BizException(ErrorCode.FILE_TOO_LARGE);
        }
        String detected;
        try (InputStream in = file.getInputStream()) {
            detected = tika.detect(in);       // 读取文件头判断真实类型
        }
        if (!ALLOWED.contains(detected)
                || !extensionMatches(file.getOriginalFilename(), detected)) {
            throw new BizException(ErrorCode.FILE_TYPE_NOT_ALLOWED);
        }
        if (detected.equals("application/pdf")) {
            pdfInspector.rejectActiveContent(file); // 含 JavaScript 则拒绝
        }
        String key = YearMonth.now(clock) + "/" + UUID.randomUUID();
        String sha256 = storage.put(key, file);    // 边写边算哈希
        return attachmentMapper.insertTemp(u.id(), refType,
            sanitize(file.getOriginalFilename()), key, detected,
            file.getSize(), sha256);
    }
}
```

限制规则：

项	限制	检查位置
类型	JPEG、PNG、PDF	前端 accept；后端 Tika
单文件	≤ 10 MB	前端、Nginx（`client_max_body_size 11m`，含表单开销）、Spring `max-file-size`、数据库 CHECK
每条报销明细	≤ 10 个票据	服务端
每张申请	≤ 50 个附件，总大小 ⩽ 100 MB	服务端，可在系统配置中调整
上传频率	每用户每分钟 ⩽ 30 次	Bucket4j 限流

上传后的附件状态为 `TEMP`，只有上传人能访问；保存草稿时才绑定到申请（`BOUND`）。绑定时只接受“本人上传的 TEMP 附件”或“本单已绑定的附件”，否则用户可以把别人的附件 ID 写进自己的草稿，借此读取他人文件。超过 24 小时仍未绑定的 TEMP 附件由定时任务删除。

下载响应带 `X-Content-Type-Options: nosniff`、`Cache-Control: private, no-store`，预览 PDF 时加 `Content-Security-Policy: sandbox`；文件名用 `filename*=UTF-8''...` 编码。

Q2：用户注册、登录时的信息是如何验证的？密码怎么存？

注册时，后端用 `RegisterRequest` 接收请求，格式由 Bean Validation 注解检查（见 4.1.4）：用户名 4–32 位、以字母开头，邮箱格式合法，密码 8–64 位且同时含字母和数字。前端使用同一组正则做即时提示。用户名和邮箱是否已被占用这类需要查库的检查在 `AuthService` 中完成，数据库上的 `lower(username)`、`lower(email)` 唯一索引兜住并发注册。

注册请求只有用户名、姓名、邮箱、密码和申请加入的部门，没有角色字段。新用户状态为 `PENDING`，不能登录，由管理员审核：

状态	含义	能否登录	转换
PENDING	已注册，待审核	否，提示“待管理员审核”	通过 → ACTIVE；拒绝 → REJECTED
ACTIVE	正常	是	停用 → DISABLED
DISABLED	停用（如离职）	否	启用 → ACTIVE
REJECTED	注册被拒绝	否	终态

审核通过时管理员确认所属部门，系统自动授予 `EMPLOYEE` 角色。部门负责人、财务审批人、企业负责人、系统管理员只能由管理员在用户管理页另行授予，操作写审计日志。

密码用 BCrypt（cost = 10）哈希后存入 `password_hash`，存储格式为 `{bcrypt}$2a$10$...`。明文不落库，也不出现在日志和接口响应中。登录失败时统一提示“用户名或密码错误”，不区分账号是否存在；同一账号 15 分钟内连续失败 5 次锁定 15 分钟。修改密码需要校验旧密码，成功后 `token_version + 1` 并撤销该用户所有刷新令牌，其他设备上的登录随之失效。

登录成功后签发两个令牌：访问令牌（JWT，30 分钟）放在请求头里；刷新令牌（7 天）在 Web 端写入 HttpOnly Cookie，在移动端放在响应体中。刷新令牌每次使用后轮换，数据库只存它的 SHA-256；已经轮换掉的旧令牌再次出现时，判定为泄露，整族令牌全部撤销。

Q3：权限是如何控制的？为什么系统管理员不能审批？

权限拆成三个互相独立的维度：

维度	回答的问题	来源
功能权限	能调用哪些接口	角色 → 权限点（RBAC）
数据范围	能看哪些申请	本人、被分配过的待办、本人负责的部门，加上管理员授予的 DEPT / ALL 范围
审批权	能批哪一张单	只看本人是否为当前节点 PENDING 待办的被分配人

如果把“审批”做成角色权限，持有该角色的人就能批任何单，满足不了“部门负责人只能处理分配给自己的任务”。所以审批权完全由流程实例决定，角色只作为审批人解析规则的输入。

系统管理员的 `SYS_ADMIN` 角色只有管理类权限点（用户、部门、授权、模板、配置、审计、转办）。服务端拒绝把 `TASK_HANDLE` 授予 `SYS_ADMIN`；审批人解析时，只持有 `SYS_ADMIN` 的用户视为无效审批人。管理员要看业务正文，必须被授予 `BIZ_CONTENT_READ`，每次访问写审计日志。这样“管理别人账号”和“审批别人单据”是两件分开授权的事。

权限矩阵（✓ 允许；A 仅当本人是该申请的被分配审批人；D 限本人负责的部门；G 需管理员显式授权；— 禁止）：

操作	员工	部门负责人	财务审批人	企业负责人	系统管理员
新建、编辑、提交、撤回本人申请	✓	✓	✓	✓	—
查看他人申请详情	—	A / D	A	A / G	G
预览、下载他人申请附件	—	A / G	A	A / G	G
查看他人草稿	—	—	—	—	—
批准、驳回	—	A	A	A	—
审批本人申请	—	—	—	—	—
本部门申请查询与统计	—	D	—	—	—
财务统计	—	—	G	G	—
企业范围查询与统计	—	—	—	G	—
用户、部门、角色、授权、模板管理	—	—	—	—	✓
转办	—	—	—	—	✓（需填原因）
修改、删除审批日志与审计日志	—	—	—	—	—
读取、标记他人通知	—	—	—	—	—

这些检查都在服务端：功能权限由 `@PreAuthorize` 检查，资源权限由 `AccessPolicy` 检查（见 4.1.6），列表的数据范围写进 SQL 条件（见 4.1.2），审批权写在批准 / 驳回的 SQL 条件里（`assignee_id = 当前用户 AND status = 'PENDING'`）。前端隐藏菜单和按钮只是为了界面整洁。

Q4：申请人、部门、金额这些字段如何防止被前端篡改？

这些字段不在请求 DTO 里（见 4.1.4），客户端在 JSON 里带上也会被丢弃，全部由服务端确定：

字段	来源
`applicant_id`	访问令牌中的当前用户
`dept_id`	申请人主属部门；提交时固化，之后部门调整不影响该单
`app_no`	前缀 + 上海时区日期 + 序列号，如 `BX20261120000123`
各类时间	数据库 `now()`
`amount_cents`	服务端计算：采购为拟选报价金额，差旅为预算，报销为明细合计，自定义为模板指定的金额字段
`status`、`current_node_seq`、`version`	只能由流程引擎修改
审批人	提交时由引擎按模板解析

审批人在详情页能看到的“当前审批人”“审批路径”也全部来自服务端快照，不由前端计算。

Q5：采购申请的报价规则如何校验？

采购申请由三部分组成：采购明细（可增删，≥ 1 条）、报价比较（≥ 2 家）、采购选择（拟选供应商与理由）。明细金额 `amount_cents = round(quantity × unit_price_cents)` 由服务端用 `BigDecimal`、`HALF_UP` 计算，数量用字符串传输，避免浮点。

提交时 `PurchaseService` 依次检查以下规则，失败时在 `errors[].rule` 中返回规则码：

规则码	规则
PUR_ITEMS_REQUIRED	明细至少 1 条，每条字段完整
PUR_QUOTES_INSUFFICIENT	非单一来源时，至少 2 家不同供应商的报价（供应商名去空白、不区分大小写比较）
PUR_QUOTE_EVIDENCE	线下报价需有报价单附件或报价明细说明；线上报价必须有购买链接和至少 1 张截图
PUR_SELECTED_INVALID	拟选报价必须属于本单
PUR_NOT_LOWEST_REASON	拟选报价不是最低价时，必须填写理由（≥ 10 字）；金额相同视为最低价
PUR_SINGLE_SOURCE	勾选单一来源时，说明 ≥ 20 字，并附说明材料或在说明中写明依据
PUR_OVER_BUDGET	拟选报价金额不超过预算上限
PUR_ARRIVAL_DATE	期望到货日期不早于提交当天

“未选最低价”的判断：

PurchaseService.java

```java
long lowest = quotes.stream()
    .mapToLong(PurchaseQuote::getQuoteAmountCents).min().orElseThrow();
boolean notLowest = selected.getQuoteAmountCents() > lowest;
if (notLowest && length(detail.getNotLowestReason()) < 10) {
    errors.add("notLowestReason", "PUR_NOT_LOWEST_REASON",
               "未选择最低报价，请说明理由（至少 10 字）");
}
```

审批界面的报价对比表中，最低价高亮、拟选行加粗，未选最低价时显示红色标签和理由；单一来源采购在页头用红色横幅显示说明与附件，审批人不会漏看。

内置采购流程：① 部门负责人审批 → ② 财务预算审查（财务审批人主办人）→ ③ 企业负责人审批（仅当金额 ≥ 5 万元）。

Q6：差旅时长如何计算？差旅单如何被报销关联？

出差天数按自然日计算：取开始、结束时间在上海时区的日期，天数 = 结束日期 − 开始日期 + 1。例如 11 月 20 日 18:00 出发、11 月 21 日 09:00 返回，记 2 天。差旅补贴、考勤、统计都按天计，晚上出发次日返回的行程实际占用了两个工作日，所以用自然日作为主口径。实际时长（小时，保留一位小数）作为辅助信息，用来区分当日往返和跨夜，上例为 15.0 小时。

TravelService.java — Java

```java
public class TravelService {
    private static final ZoneId CN = ZoneId.of("Asia/Shanghai");

    void computeDuration(TravelDetail t) {
        LocalDate d1 = t.getStartTime().atZoneSameInstant(CN).toLocalDate();
        LocalDate d2 = t.getEndTime().atZoneSameInstant(CN).toLocalDate();
        t.setNaturalDays((int) ChronoUnit.DAYS.between(d1, d2) + 1);

        long minutes = Duration.between(t.getStartTime(),
                                        t.getEndTime()).toMinutes();
        t.setDurationHours(BigDecimal.valueOf(minutes)
            .divide(BigDecimal.valueOf(60), 1, RoundingMode.HALF_UP));
    }
}
```

两个值都在保存和提交时由服务端计算落库，客户端传值被忽略；界面显示为“2 天（自然日）/ 实际 15.0 小时”。

其他校验：结束时间不得早于开始时间，跨度不超过 90 天；预算不得为负；同行人员可选系统用户或填写姓名，系统用户必须是启用状态、不能是本人、不能重复。开始时间早于提交时间时不拦截，审批界面显示“事后补报”标签。

本人的、已通过的差旅单可在报销单的“关联申请”中选择。报销详情页用卡片显示差旅单编号、目的地、起止时间、天数和预算，点击可跳转。有权查看报销单的审批人，对它关联的差旅单获得只读查看权（摘要与流程结果，不含附件），由 `AccessPolicy.canViewAsLinked` 判断。

内置差旅流程：① 部门负责人审批 → ② 企业负责人审批（仅当预算 ≥ 2 万元或天数 > 7）。

Q7：报销金额和票据如何校验？

金额在前端以字符串输入，用 `yuanToCents` 转成整数分（见 4.2.1），接口只传分。提交时服务端用明细金额重新求和，覆盖客户端可能传来的任何合计值：

ExpenseService.java — Java

```java
public class ExpenseService {
    long recomputeTotal(List<ExpenseItem> items) {
        long total = 0;
        for (ExpenseItem it : items) {
            // 溢出时抛 ArithmeticException，不会悄悄变成负数
            total = Math.addExact(total, it.getAmountCents());
        }
        return total;
    }

    void beforeSubmit(Application app, ExpenseDetail d,
                      List<ExpenseItem> items) {
        long total = recomputeTotal(items);
        d.setTotalAmountCents(total);
        app.setAmountCents(total);   // 统计和审批路径都用这个值
    }
}
```

报销单提交时至少有 1 条有效明细（界面默认展示 3 行），校验规则：

规则码	规则
EXP_EMPTY_ITEM	不允许全空的明细行。前端提交前会剔除空行；服务端如果仍收到，直接报错，不替前端静默丢弃
EXP_ITEM_TYPE / EXP_ITEM_DATE / EXP_ITEM_AMOUNT	费用类型必须为启用的类型；发生日期不晚于当天且不早于 365 天前；金额 > 0 且单条 ≤ 10 万元
EXP_INVOICE_REQUIRED	非无票明细至少挂 1 张票据
EXP_NO_INVOICE_REASON	勾选“无票”时必须填写原因（≥ 10 字），且不能同时挂票据
EXP_DUP_INVOICE_IN_APP	同一报销单内，同一文件（SHA-256 相同）或同一票据号（去空格、转大写后相同）不能被使用两次
EXP_LINK_INVALID	关联申请必须是本人的、已通过的差旅或采购单；不关联时必须填写独立报销说明
EXP_FILE_TYPE / EXP_FILE_SIZE	见 Q1 的限制规则

同单票据去重在数据库层有唯一索引兜底：

V1__init_schema.sql — SQL

```sql
CREATE UNIQUE INDEX uk_invoice_app_sha
    ON expense_invoice (application_id, file_sha256);

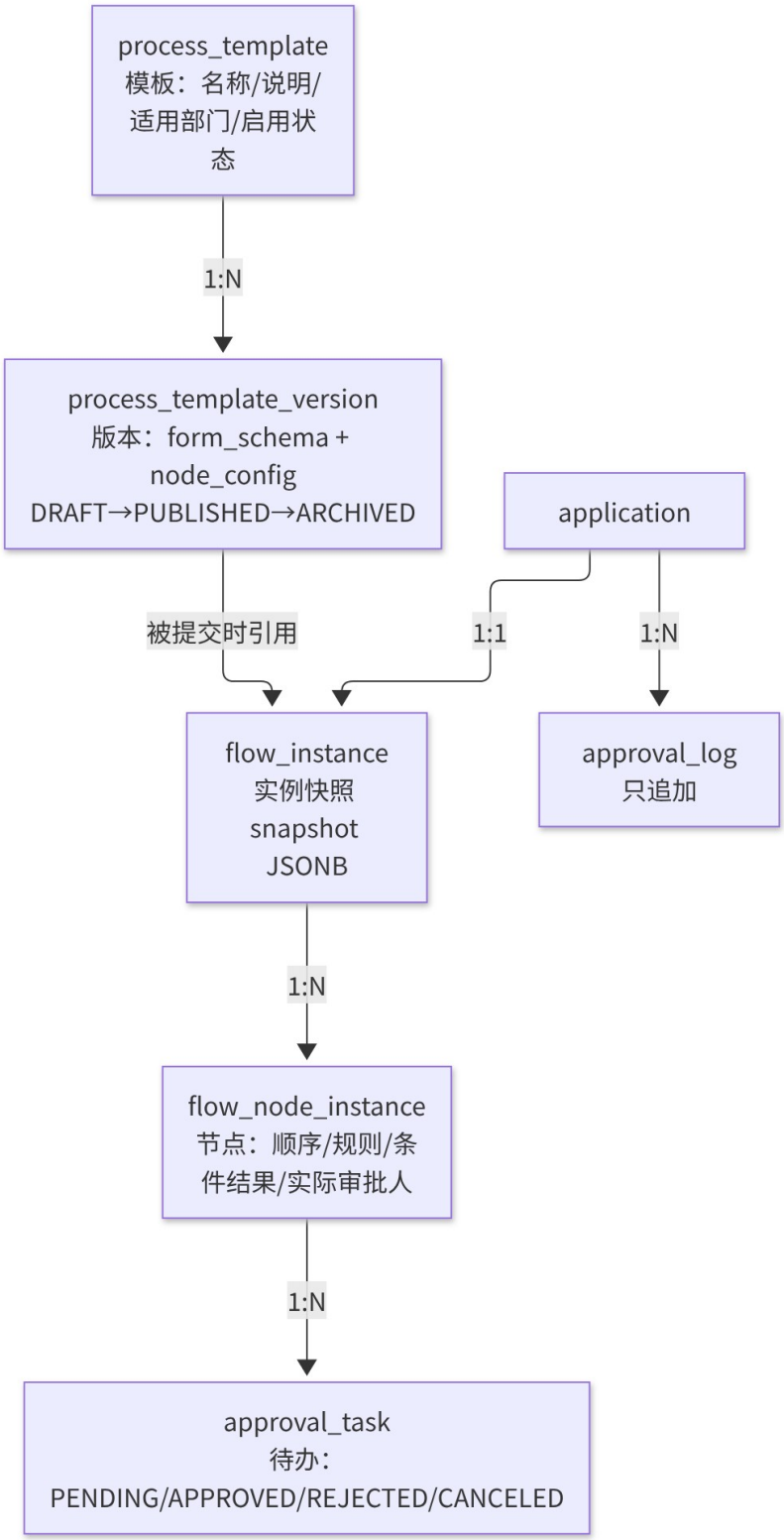
CREATE UNIQUE INDEX uk_invoice_app_no
    ON expense_invoice (application_id, invoice_no_norm)
 WHERE invoice_no_norm IS NOT NULL;
```

费用类型预置交通、住宿、餐饮、办公、其他五种，管理员可新增和停用；已被引用的类型不能删除。财务审核界面中，无票明细整行橙色高亮，页头显示“含 N 条无票支出（合计 ¥X）”。

内置报销流程：① 部门负责人审批 → ② 财务审核（财务审批人主办人）→ ③ 企业负责人审批（仅当金额 ≥ 1 万元）。

Q8：管理员如何自定义审批流程？

一个流程模板由“表单字段”和“审批节点”两部分组成，按版本保存：



采购、差旅、报销三类内置业务同样以模板形式存在（`category = BUILTIN`），表单字段固定，管理员只能调整节点配置（如金额阈值、审批人规则）。自定义模板（`category = CUSTOM`）的表单和节点都可配置。两者走同一个流程引擎。

模板基本属性：名称（2–50 字）、编码（创建后不可改）、说明、适用部门（为空表示全部部门）、启用状态。停用后员工不能新建，已有草稿提交时被拒绝，审批中的申请不受影响。

字段类型与可配置的校验：

类型	渲染组件	可配置校验	值的格式
TEXT 单行文本	el-input	必填、最小 / 最大长度（≤ 200）、正则（预设手机号、邮箱，或自定义）	字符串
TEXTAREA 多行文本	el-input type=textarea	必填、最小 / 最大长度（≤ 2000）	字符串
NUMBER 数字	el-input-number	必填、最小值、最大值、小数位数（0–4）、步长	十进制字符串，如 "3.5"
AMOUNT 金额	AmountInput	必填、最小 / 最大金额；可标记为“统计金额”（至多一个字段）	整数分
DATE 日期	el-date-picker	必填、最早 / 最晚日期（可写 TODAY、TODAY+30）、不早于另一日期字段	YYYY-MM-DD
RADIO 单选	el-radio-group	必填、选项（2–20 项，值唯一）	选项值
ATTACHMENT 附件	AttachmentUploader	必填、最多个数（1–10）、允许的类型、单个大小（≤ 10 MB）	附件 ID 数组

form_schema 示例（请假申请，节选）：

form_schema.json JSON

```json
{
  "fields": [
    {"key": "leaveType", "label": "请假类型", "type": "RADIO",
     "required": true,
     "options": [
       {"value": "ANNUAL",   "label": "年假"},
       {"value": "PERSONAL", "label": "事假"},
       {"value": "SICK",     "label": "病假"}]},
    {"key": "startDate", "label": "开始日期", "type": "DATE",
     "required": true},
    {"key": "endDate", "label": "结束日期", "type": "DATE",
     "required": true, "rules": {"notBefore": "startDate"}},
    {"key": "days", "label": "请假天数", "type": "NUMBER",
     "required": true,
     "rules": {"min": "0.5", "max": "30", "precision": 1, "step": "0.5"}},
    {"key": "reason", "label": "请假事由", "type": "TEXTAREA",
     "required": true, "rules": {"minLength": 5, "maxLength": 500}}
  ]
}
```

节点按数组顺序依次审批，设计器里用上移 / 下移或拖拽调整顺序。每个节点配置名称、审批人规则和可选的生效条件：

审批人规则	解析方式
SPECIFIED_USER 指定用户	直接使用配置的用户，要求启用状态
APPLICANT_DEPT_HEAD 申请人部门负责人	申请人主属部门的 head_user_id
ROLE 角色主办人（扩展规则）	持有该角色且标记为主办人的用户，如财务审批人、企业负责人

生效条件可以引用数字、金额、日期、单选字段，内置模板还可以引用系统变量 amountCents、naturalDays；运算符为等于、不等于、大于、大于等于、小于、小于等于、属于。条件不满足的节点在实例中标记为 SKIPPED。

node_config.json JSON

```
{
  "nodes": [
    {"nodeKey": "N1", "name": "部门负责人审批",
     "approverRule": {"type": "APPLICANT_DEPT_HEAD"},
     "fallbackRule": {"type": "ROLE", "roleCode": "ENTERPRISE_HEAD"}},
    {"nodeKey": "N2", "name": "企业负责人审批",
     "approverRule": {"type": "SPECIFIED_USER", "userId": 2},
     "condition": {
       "logic": "AND",
       "rules": [{"var": "days", "op": "GT", "value": "3"}]}}
  ]
}
```

发布前由 TemplatePublishValidator 校验，一次返回全部问题，设计器逐项定位：

校验码	内容
TPL_BASIC	名称、编码、说明合法；适用部门存在
TPL_FIELD_KEY_DUP	字段标识唯一（不区分大小写再比一次，防止 Days 与 days 混淆），不与保留字冲突
TPL_FIELD_INVALID	校验规则与字段类型匹配（如选项只能用于单选）；最小值 ≤ 最大值；正则可编译；“不早于”引用的字段存在且为日期
TPL_FIELD_EMPTY	自定义模板至少 1 个字段
TPL_NODE_COUNT	至少 2 个顺序节点，最多 10 个
TPL_NODE_INCOMPLETE	每个节点名称非空、审批人规则完整
TPL_APPROVER_INVALID	指定用户存在且启用，且不是只有管理员角色的账号；部门负责人规则要求适用部门都已设置负责人；角色规则要求该角色当前有主办人
TPL_CONDITION_REF	条件引用的字段存在，类型与运算符匹配，值可解析
TPL_ALL_SKIPPABLE	至少一个节点无条件，否则可能出现所有节点都被跳过
TPL_CYCLE	节点序号连续、无重复，按序号建图后拓扑排序无环

版本规则：

1. 已发布、已归档版本的表单和节点配置不可修改。服务端拒绝，数据库触发器也拒绝更新这两列。
2. 编辑模板时，复制当前发布版本生成新的 DRAFT 版本，每个模板同一时刻至多一个草稿版本。
3. 发布时新版本变为 PUBLISHED，旧版本变为 ARCHIVED，新申请使用新版本。发布必须填写变更说明，写入审计日志。
4. 申请提交时绑定当时的发布版本并生成快照（见 Q9），之后模板、阈值、部门负责人的任何变化都不改写已提交的申请。

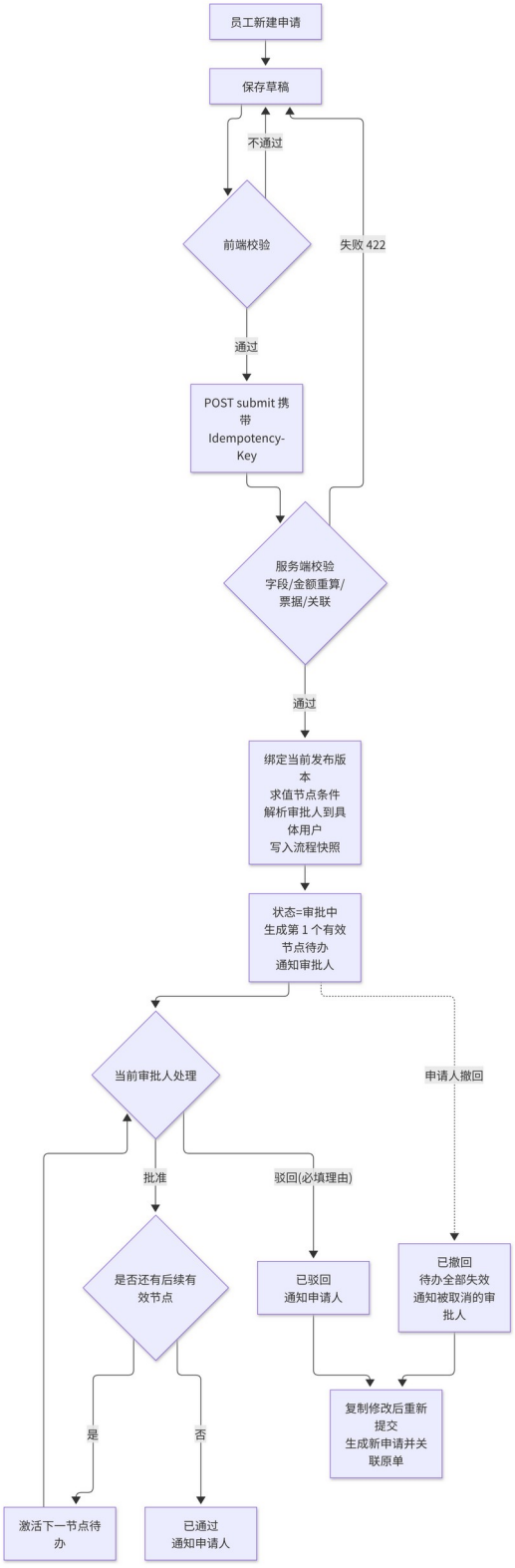
5. 自定义申请的草稿如果是按旧版本填写的，提交时返回 `40906`，前端按新版本重新渲染表单，同名同类型字段的值保留，用户确认后再提交。

员工端由 `FormRenderer` 按 `form_schema` 渲染表单。服务端提交校验使用同一份 Schema，由 `FormSchemaValidator` 逐字段检查，Schema 中未定义的字段一律拒绝。

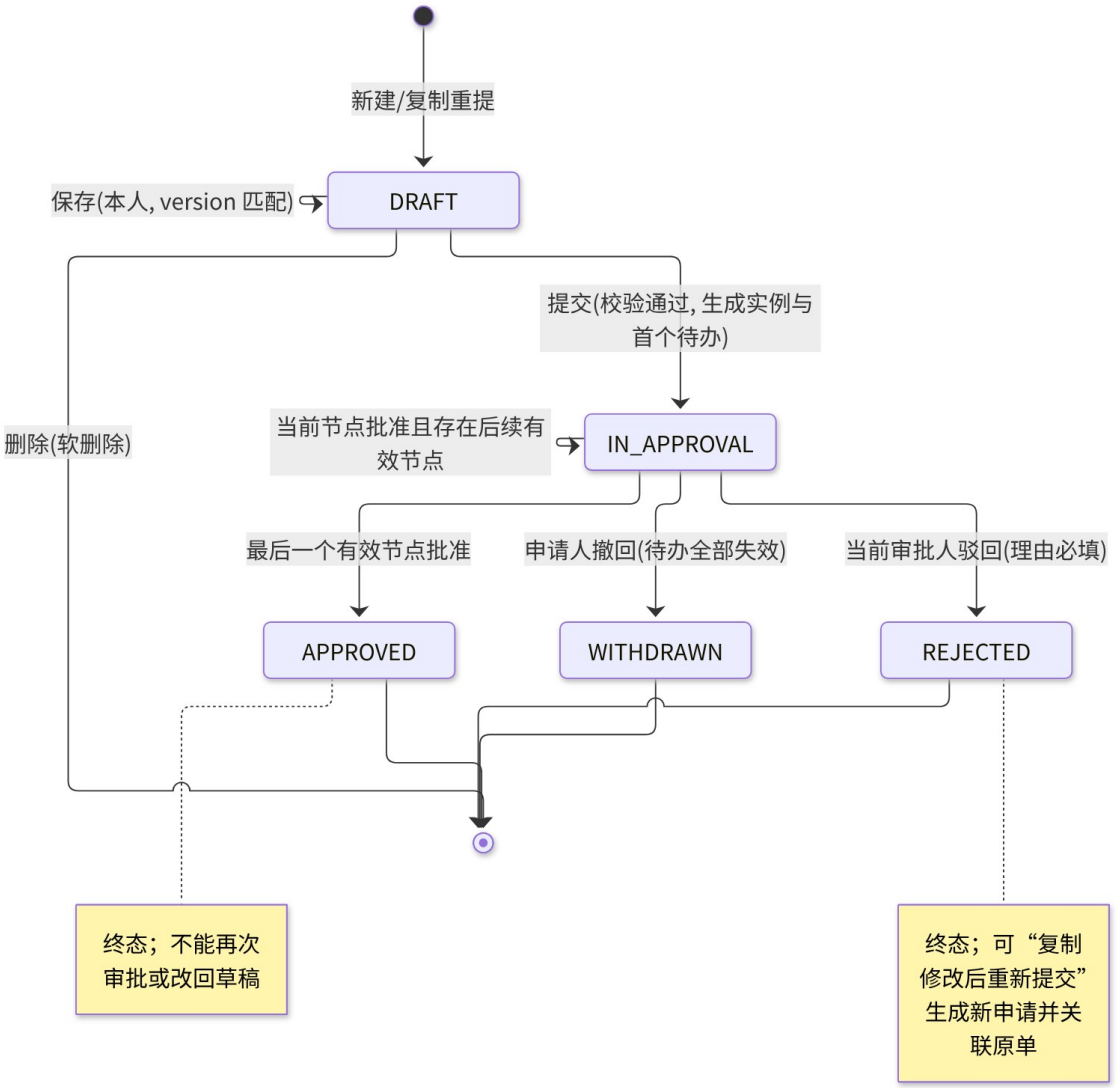
初始化数据中带两个自定义模板：请假申请（部门负责人 → 企业负责人，请假超过 3 天时）、用章申请（部门负责人 → 印章管理员）。

Q9：审批流程是如何流转的？审批人如何确定？

整体流程：


员工新建申请
保存草稿
不通过
前端校验
失败 422
通过
POST submit 携带 Idempotency-Key
服务端校验 字段/金额重算/票据/关联
通过
绑定当前发布版本 求值节点条件 解析审批人到具体用户 写入流程快照
状态=审批中 生成第 1 个有效节点待办 通知审批人
当前审批人处理
申请人撤回
批准
驳回(必填理由)
是否还有后续有效节点
已驳回 通知申请人
已撤回 待办全部失效 通知被取消的审批人
复制修改后重新提交 生成新申请并关联原单
是
否
激活下一节点待办
已通过 通知申请人

申请状态机：



当前状态	操作	前提	新状态
—	新建	模板启用、适用于申请人部门	草稿
草稿	保存	本人，`version` 一致	草稿
草稿	提交	本人，完整校验通过，所有节点审批人可解析	审批中
审批中	批准	本人是当前待办的被分配人，且不是申请人	审批中 / 已通过
审批中	驳回	同上，理由 5–500 字	已驳回
审批中	撤回	本人	已撤回
已通过 / 已驳回 / 已撤回	任何变更	—	拒绝，返回 `40901`

提交时，引擎在同一个事务里为每个条件成立的节点解析出唯一的具体审批人，写入 `flow_node_instance` 和 `flow_instance.snapshot`，只为第一个有效节点创建待办。前一节点通过前，后续节点没有待办，也就无法被审批。

之所以提交时一次解析所有节点，而不是轮到该节点时再解析：需求要求后续人员调整不改写已有实例。代价是审批途中有人离职，待办仍挂在原审批人名下，需要管理员转办（见 Q11）。

禁止自审：解析结果是申请人本人时，按节点的 `fallbackRule` 再解析（默认为企业负责人）；仍是本人，则使用系统配置的兜底审批人（初始化为财务部负责人）；还是本人就提交失败。替代过程写入快照，详情页显示为“申请人为本部门负责人，按替代规则由企业负责人审批”。批准 / 驳回的 SQL 中还会再检查一次 `applicant_id <> 当前用户`。

同一个人出现在相邻两个节点时（如财务部负责人既是部门负责人又是财务主办人），两个节点都保留、分别审批，不自动合并。合并会让日志少一条“财务审核”记录，审计时说不清这一步是否审过。

快照示例：

flow_instance.snapshot — JSON

```
{
  "templateCode": "BUILTIN_EXPENSE",
  "versionNo": 1,
  "resolvedAt": "2026-11-20T10:03:11+08:00",
  "variables": {"amountCents": 1268000},
  "nodes": [
    {"seq": 1, "nodeKey": "N1", "name": "部门负责人审批",
     "rule": {"type": "APPLICANT_DEPT_HEAD"}, "effective": true,
     "assignee": {"id": 3, "name": "李研", "deptName": "研发部"},
     "resolveNote": "研发部负责人"},
    {"seq": 2, "nodeKey": "N2", "name": "财务审核",
     "rule": {"type": "ROLE", "roleCode": "FINANCE_APPROVER"},
     "effective": true,
     "assignee": {"id": 9, "name": "吴审", "deptName": "财务部"},
     "resolveNote": "财务审批人主办人"},
    {"seq": 3, "nodeKey": "N3", "name": "企业负责人审批",
     "rule": {"type": "ROLE", "roleCode": "ENTERPRISE_HEAD"},
     "condition": {"logic": "AND", "rules": [
       {"var": "amountCents", "op": "GTE", "value": "1000000"}]},
     "conditionResult": true, "effective": true,
     "assignee": {"id": 2, "name": "赵总", "deptName": "总经办"},
     "resolveNote": "企业负责人主办人"}
  ]
}
```

节点状态为 `WAITING → ACTIVE → APPROVED / REJECTED / CANCELED`，另有 `SKIPPED`（条件不满足）和 `NOT_REACHED`（驳回后未到达）；同一实例任一时刻至多一个 `ACTIVE` 节点，由部分唯一索引保证。待办状态为 `PENDING → APPROVED / REJECTED / CANCELED / TRANSFERRED`。

Q10：撤回和审批同时发生怎么办？重复点击会不会重复流转？

三道防线：

1. 幂等键。提交、批准、驳回、撤回、催办接口必须带请求头 `Idempotency-Key`。前端在用户打开确认框时生成，网络超时重试和重复点击都复用同一个键。服务端先插入 `idempotency_record`：插入成功才执行业务，并在同一事务里保存响应；键已存在且已完成，

直接返回保存的响应；仍在处理中返回 `40903`；同一个键对应不同请求体返回 `40904`。业务失败回滚时幂等记录一起回滚，用户修正后可以重试。

2. 条件更新。`application.version` 是并发锚点，所有状态变更都写成 `WHERE status = ? AND version = ?` 的条件更新，影响 0 行就说明状态已被别人改过，事务回滚并返回 `40901`。
3. 唯一约束兜底。`approval_log(task_id, action)` 唯一；每个节点至多一个 PENDING 待办；`notification(recipient_id, dedup_key)` 唯一。

批准的 SQL 序列：

批准：SQL 序列 SQL

```sql
-- ① 锁定并推进申请：状态、版本、当前节点都要对得上
UPDATE application
   SET version          = version + 1,
       current_node_seq = :nextSeq,       -- 无后续有效节点时为 NULL
       status      = CASE WHEN :isLast THEN 'APPROVED'
                          ELSE 'IN_APPROVAL' END,
       finished_at = CASE WHEN :isLast THEN now() END,
       updated_at  = now()
 WHERE id = :appId
   AND status = 'IN_APPROVAL'
   AND version = :appVersion
   AND current_node_seq = :taskNodeSeq
   AND applicant_id <> :uid;              -- 禁止自审
-- 影响 0 行 → 回滚，返回 40901 STATE_CHANGED

-- ② 完成任务：只能是本人的 PENDING 任务
UPDATE approval_task
   SET status = 'APPROVED', comment = :comment, acted_at = now()
 WHERE id = :taskId
   AND assignee_id = :uid
   AND status = 'PENDING';
-- 影响 0 行 → 回滚，返回 40902 TASK_ALREADY_PROCESSED

```

批准：SQL 序列（续）

```sql
-- ③ 结束当前节点，激活下一节点并创建待办
UPDATE flow_node_instance
   SET status = 'APPROVED', finished_at = now()
 WHERE id = :nodeId AND status = 'ACTIVE';

UPDATE flow_node_instance
   SET status = 'ACTIVE', activated_at = now()
 WHERE flow_instance_id = :fiId AND seq = :nextSeq
   AND status = 'WAITING';

INSERT INTO approval_task
       (application_id, node_instance_id, assignee_id, status)
VALUES (:appId, :nextNodeId, :nextAssigneeId, 'PENDING');

-- ④ 同一事务内写日志与通知
INSERT INTO approval_log (..., action, comment, result_status)
VALUES (..., 'APPROVE', :comment, :newStatus);

INSERT INTO notification
       (recipient_id, type, title, application_id, dedup_key)
VALUES (:nextAssigneeId, 'TASK_ASSIGNED', ..., :appId,
        'TASK_ASSIGNED:' || :newTaskId)
ON CONFLICT (recipient_id, dedup_key) DO NOTHING;
```

撤回的 SQL 序列：

撤回：SQL 序列

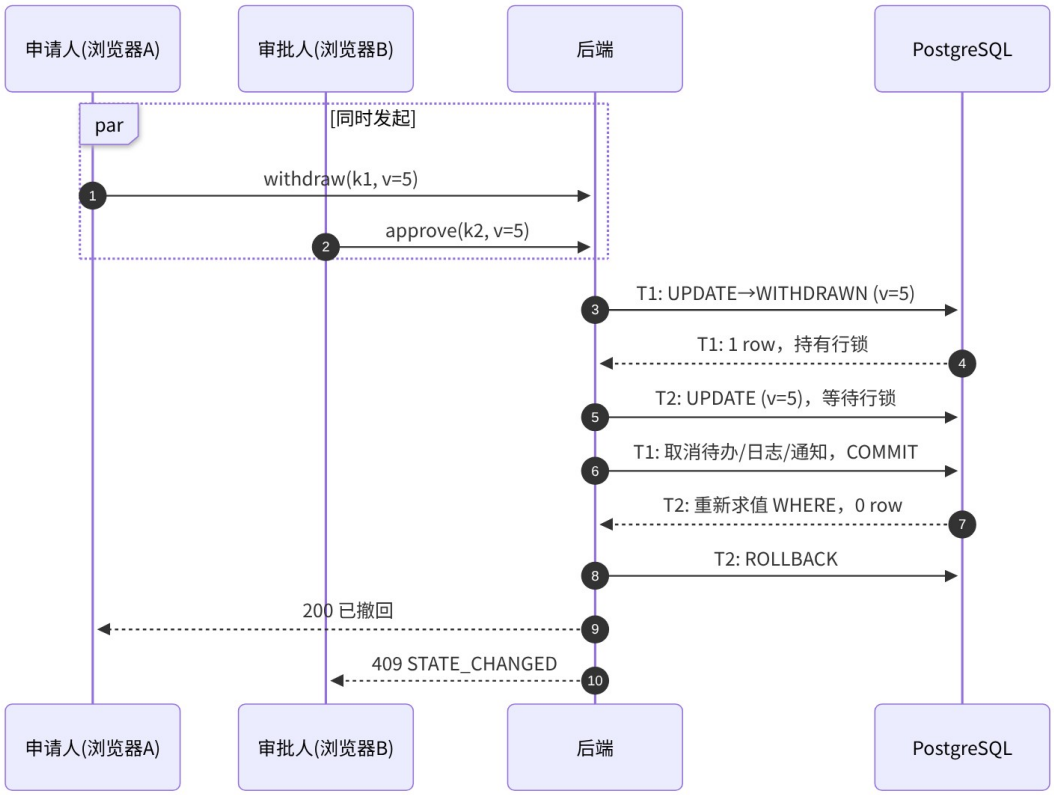
```sql
UPDATE application
   SET status = 'WITHDRAWN', version = version + 1,
       finished_at = now(), updated_at = now()
 WHERE id = :appId
   AND applicant_id = :uid
   AND status = 'IN_APPROVAL'
   AND version = :appVersion;
-- 影响 0 行 → 40901（例如审批人刚批准了最后一个节点）

-- 返回被取消的待办，用于通知对应审批人
UPDATE approval_task
   SET status = 'CANCELED', acted_at = now()
 WHERE application_id = :appId AND status = 'PENDING'
RETURNING id, assignee_id;

UPDATE flow_node_instance
   SET status = 'CANCELED', finished_at = now()
 WHERE flow_instance_id = :fiId
   AND status IN ('ACTIVE', 'WAITING');

INSERT INTO approval_log (..., action, result_status)
VALUES (..., 'WITHDRAW', 'WITHDRAWN');
```

撤回与批准同时到达时的时序：



T1 为撤回事务，T2 为批准事务，两者竞争同一 `application` 行。隔离级别保持 PostgreSQL 默认的 `READ COMMITTED`：T2 等待行锁，T1 提交后 T2 在新的行版本上重新求值 `WHERE status = 'IN_APPROVAL' AND version = 5`，结果为假，影响 0 行而回滚。如果批准先拿到行锁，结果对称：批准成功，撤回返回 409。前端收到 409 时弹窗提示“申请状态已变化”，并提供刷新按钮。

没有用 `SELECT … FOR UPDATE`：条件更新一条语句同时完成检查和修改，少一次往返；`version` 本来就要返回给前端判断“是否基于最新状态操作”，两件事用同一个字段解决。客户端即使伪造 `appVersion`，条件更新仍以数据库当前行为准，最多让自己的请求失败。

几种重复情况的结果：

1. 同一用户连点两次：按钮在请求期间禁用；即使发出两次请求，幂等键相同，第二次返回第一次的结果。
2. 同一用户在两个浏览器各点一次（幂等键不同）：两个事务竞争同一行，后到的影响 0 行，返回 `40901`。待办、日志、通知各只有一份。
3. 响应在网络上丢失：客户端用同一幂等键重试，拿到第一次的成功响应。

Q11：驳回后如何重新提交？审批人离职怎么办？

驳回理由必填（5–500 字）。驳回即结束本次审批，后续节点标记为 `NOT_REACHED`，申请人收到通知，详情页用红色标出驳回节点、驳回人和理由。

已驳回、已撤回都是终态，不能改回草稿。申请人在详情页点击“复制并修改”，生成一张新草稿（新编号），source_application_id 指向原单，业务字段和附件引用一并复制。新单提交时按当时的模板版本重新解析流程。两张单的详情页互相显示“来源单 / 后续单”链接，原单的审批历史保持不变。

审批人被停用时，其名下的待办由管理员转办：选择新审批人（不能是申请人，不能是只有管理员角色的账号）并填写原因。原待办变为 TRANSFERRED，为新审批人创建 PENDING 待办，节点实例的审批人更新，并在 assignee_history 中追加记录；同时写审批日志 TRANSFER 和审计日志，通知新旧审批人与申请人。这是唯一能修改已提交实例审批人的途径。

Q12：审批日志和管理变更如何留痕、防止篡改？

审批日志 approval_log 记录每一次提交、批准、驳回、撤回、转办：操作人（含姓名快照）、动作、意见或驳回理由、操作前后的申请状态、申请版本（第几次提交）、模板版本号、时间、IP。详情页按时间顺序展示。

管理变更写入 audit_log：注册审核、角色授予与回收、主办人设置、部门负责人变更、数据范围授权与回收、模板发布与启停、系统配置修改、转办、管理员查看业务正文、账号锁定。每条记录保存操作人、目标对象、修改前后的值（JSONB）、原因和 IP。管理端写操作由 @Audited 切面统一记录，管理端审计页可按对象查询，并以 JSON 差异对比显示前后值。

两张表只允许追加，靠两层保证：

V2__append_only_triggers.sql

```sql
-- 触发器：任何 UPDATE / DELETE 都报错
CREATE FUNCTION forbid_change() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION '% is append-only', TG_TABLE_NAME;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_approval_log_append_only
    BEFORE UPDATE OR DELETE ON approval_log
    FOR EACH ROW EXECUTE FUNCTION forbid_change();

CREATE TRIGGER trg_audit_log_append_only
    BEFORE UPDATE OR DELETE ON audit_log
    FOR EACH ROW EXECUTE FUNCTION forbid_change();

-- 运行账号 eas_app 对日志表只有 INSERT 和 SELECT
REVOKE UPDATE, DELETE ON approval_log, audit_log FROM eas_app;
```

表属主 eas_owner 只在 Flyway 迁移时使用，应用运行时用 eas_app 连接数据库。即使应用代码有缺陷，也改不了日志。

Q13：站内通知和实时提醒是如何实现的？

通知在业务事务内写入 `notification` 表，服务重启后仍可查询。触发事件与接收人：

事件	通知类型	接收人	去重键	邮件
生成新待办（提交或上一节点通过后）	TASK_ASSIGNED	新待办的审批人	TASK_ASSIGNED:{taskId}	✓
中间节点通过	NODE_APPROVED	申请人	NODE_APPROVED:{taskId}	
最终通过	APP_APPROVED	申请人	APP_APPROVED:{appId}	✓
驳回	APP_REJECTED	申请人	APP_REJECTED:{appId}	✓
撤回	TASK_CANCELED	被取消待办的审批人	TASK_CANCELED:{taskId}	✓
转办	TASK_TRANSFERRED	原、新审批人与申请人	TASK_TRANSFERRED:{taskId}:{userId}	✓
注册审核通过	REGISTRATION_APPROVED	新用户	REG_APPROVED:{userId}	✓

去重靠 `UNIQUE(recipient_id, dedup_key)` 加 `ON CONFLICT DO NOTHING`。去重键由业务实体 ID 构成，与请求无关，请求重试、幂等重放都不会产生第二条通知。

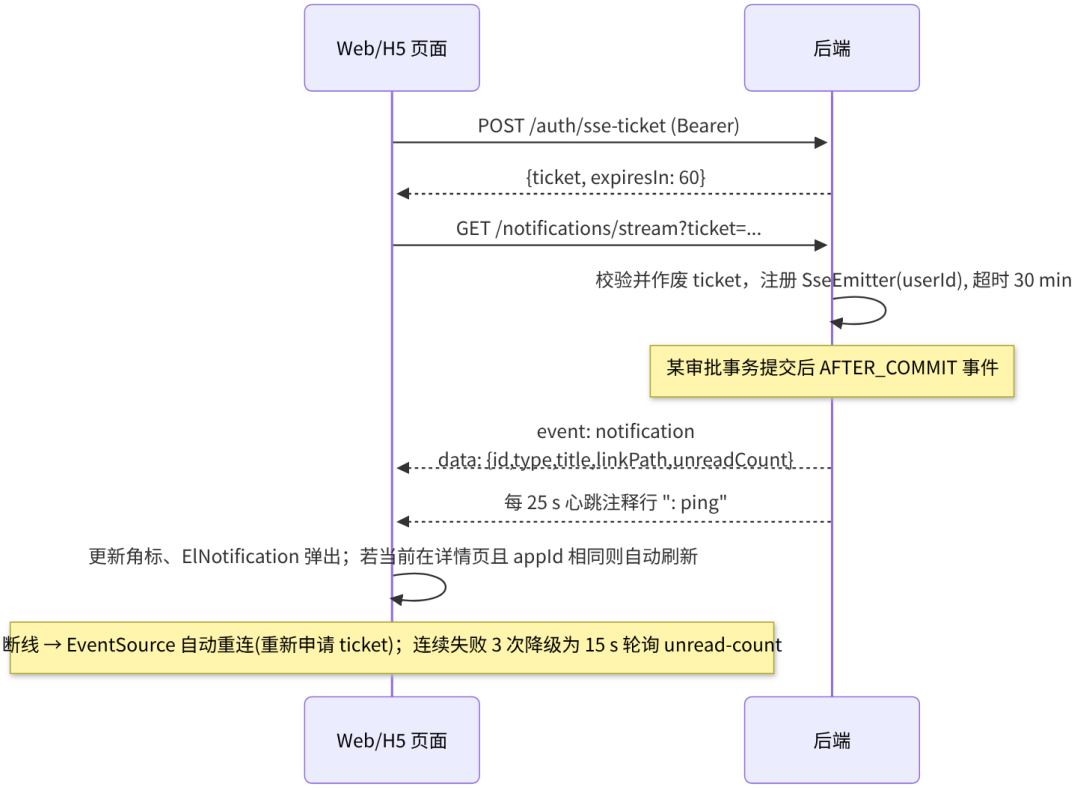
未读数、标记已读的 SQL 都带 `recipient_id = 当前用户`：

NotificationMapper.xml

```sql
-- 标记已读：只能标记自己的通知
UPDATE notification
   SET is_read = true, read_at = now()
 WHERE id = :id AND recipient_id = :uid AND NOT is_read;
-- 影响 0 行时，再查一次是否属于本人，否则返回 404
```

点击通知时，前端先标记已读，再跳转到 `link_path`（如 `/applications/305`）。详情页照常鉴权，权限已变化时显示“无权查看或已不存在”。

页面打开时用 SSE 实时推送：



推送在事务提交之后进行，事务回滚就不会推送：

NotificationPusher.java — Java

```java
@Component
@RequiredArgsConstructor
public class NotificationPusher {
    private final SseHub hub;

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onCreated(NotificationCreated e) {
        // 只推给接收人自己的连接
        hub.send(e.recipientId(), "notification", e.payload());
    }
}
```

`EventSource` 不能设置请求头，所以先用访问令牌换一个一次性票据（60 秒有效，用一次即作废，绑定用户），再用票据建立连接，访问令牌不出现在 URL 中。Nginx 对 SSE 路径关闭缓冲（`proxy_buffering off`，`proxy_read_timeout 1h`）。实时推送只是提醒，数据以接口查询为准；推送丢失时，轮询和页面切回时的拉取会补齐。

微信小程序不支持 `EventSource`，改为页面 `onShow` 时立即拉取，前台期间每 15 秒请求一次 `/notifications/changes?since=上次最大 ID`，`onHide` 时停止；有新通知时弹出 Toast 并更新“消息”tab 的角标。

页面关闭后靠邮件提醒。业务事务内同时写一条 `notification_delivery(status = PENDING)`，`MailDispatcher` 每 10 秒取一批待发记录（`FOR UPDATE SKIP LOCKED`，每批 50 条）发送；失败按 1、2、4、8、16 分钟退避重试，5 次后标记失败，站内通知不受影响。邮件只包含标题、申请编号、类型、状态、操作人、时间和“在系统中查看”链接，不包含金额明细和票据。用户可在个人中心关闭邮件提醒。演示环境用 Mailpit 收件，Web 界面在 8025 端口。

平台	页面打开时	页面关闭后
Web	SSE，失败降级为轮询	邮件
H5（手机浏览器、微信内置浏览器）	SSE，失败降级为轮询	邮件
微信小程序	前台 15 秒轮询 + onShow 拉取	邮件

Q14：统计数据的口径是什么？

口径项	定义
时间	按提交时间 `submitted_at` 落入查询区间，日期按上海时区计算；草稿没有提交时间，不进入部门和企业统计
部门	按申请提交时固化的部门，与申请人现在所在部门无关
金额	`application.amount_cents`：采购为拟选报价金额，差旅为预算，报销为明细合计，自定义为模板指定的统计金额字段
已通过金额	只统计已通过的申请，不含草稿、审批中、已驳回、已撤回
在途金额	审批中的申请金额之和，单独显示，不与已通过金额相加
申请数量	非草稿申请计数；重新提交生成的新单单独计数，原驳回单仍计入驳回数
通过率	已通过数 /（已通过数 + 已驳回数），撤回和审批中不计入分母
平均审批时长	已通过或已驳回申请的 `finished_at − submitted_at` 平均值，单位小时

统计内容与权限：

统计	使用者	内容	图表
个人	所有用户	我的申请按状态计数、我的待办数、我的已通过金额	环形图、数字卡片
本部门	部门负责人	各类申请数量与状态分布、已通过金额、在途金额、通过率、平均审批时长、月度趋势	堆叠柱状图、饼图、折线图
财务	需授权 `FIN_STATS_READ`	按费用类型的报销已通过金额、无票支出金额与占比、各部门报销与采购金额	柱状图、表格
企业	需授权 `APP_READ@ALL`	各部门对比	分组柱状图

本部门统计 SQL：

StatsMapper.xml

```sql
-- 状态分布与金额
-- :fromTs/:toTs 由上海时区日期换算，区间 [from 00:00, to+1 00:00)
SELECT a.type,
       a.status,
       count(*) AS cnt,
       sum(a.amount_cents) FILTER (WHERE a.status = 'APPROVED')
           AS approved_amount_cents,
       sum(a.amount_cents) FILTER (WHERE a.status = 'IN_APPROVAL')
           AS in_progress_amount_cents
  FROM application a
 WHERE a.dept_id = :deptId
   AND a.status <> 'DRAFT'
   AND a.deleted_at IS NULL
   AND a.submitted_at >= :fromTs
   AND a.submitted_at <  :toTs
 GROUP BY a.type, a.status;

-- 月度已通过金额趋势（按提交月份）
SELECT to_char(a.submitted_at AT TIME ZONE 'Asia/Shanghai', 'YYYY-MM')
           AS month,
       sum(a.amount_cents) AS approved_amount_cents
  FROM application a
 WHERE a.dept_id = :deptId
   AND a.status = 'APPROVED'
   AND a.submitted_at >= :fromTs
   AND a.submitted_at <  :toTs
 GROUP BY 1
 ORDER BY 1;
```

统计页顶部固定显示口径说明：“按提交时间统计；已通过金额仅含已通过申请；部门以提交时所属部门为准”。

Q15：移动端如何适配？

移动端用 uni-app 编译为 H5 和微信小程序，与 Web 端共用 `lib` 下的接口、类型和金额函数，页面单独实现（页面清单见 4.2.5）。

1. 布局使用 `rpx` 与 flex，基准宽度 750rpx，正文不出现横向滚动。Web 端的表格在手机上改为卡片列表，长表单改为分步填写。
2. 触控目标不小于 88rpx（约 44 px）。金额输入使用 `type="digit"` 调起数字键盘，仍按字符串处理两位小数；日期用原生选择器。
3. 票据上传：图片用 `uni.chooseImage`，可拍照或选相册，`sizeType` 选压缩；小程序选 PDF 用 `wx.chooseMessageFile`；H5 用 `<input type="file" accept="image/*,application/pdf">`。超过

10 MB 的图片先用 `uni.compressImage` 压缩，PDF 超限直接提示。上传有进度条，失败可重试。

4. 底部固定操作栏留出 `env(safe-area-inset-bottom)`。
5. 实时提醒：H5 用 SSE，小程序用前台轮询（见 Q13）。收到状态变化时，若正在看这张单的详情页则自动刷新，列表页顶部提示“有 N 条更新”。
6. 令牌：H5 存 `sessionStorage`；小程序存 `uni.setStorageSync`，刷新令牌在响应体中返回。

小程序正式发布需要 HTTPS 合法域名，演示时以 H5 为主（手机浏览器访问部署地址 `/m/`），小程序在开发者工具中演示。

Q16：系统如何部署和初始化?

在项目根目录执行 `docker compose up -d --build`，启动以下容器：

服务	镜像	说明
postgres	`postgres:16-alpine`	数据卷 `pgdata`；初始化脚本创建运行账号 `eas_app`
minio	`minio/minio`	数据卷 `miniodata`
minio-init	`minio/mc`	一次性任务：创建私有桶 `eas-files`
mailpit	`axllent/mailpit`	演示用 SMTP 与收件箱（8025）
backend	由 `backend/Dockerfile` 构建	启动时执行 Flyway 迁移
nginx	由 `nginx/Dockerfile` 构建	多阶段构建 Web 与 H5，托管静态资源并反向代理 `/api/`

`docker-compose.yml` 中 backend 部分：

docker-compose.yml YAML

```yaml
backend:
  build: ./backend
  depends_on:
    postgres:   { condition: service_healthy }
    minio-init: { condition: service_completed_successfully }
  environment:
    SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-demo}
    DB_URL: jdbc:postgresql://postgres:5432/eas
    DB_USER: eas_app                  # 运行账号：仅 DML 权限
    DB_PASSWORD: ${DB_PASSWORD}
    FLYWAY_USER: eas_owner            # 迁移账号
    FLYWAY_PASSWORD: ${DB_OWNER_PASSWORD}
    JWT_SECRET: ${JWT_SECRET:?set JWT_SECRET (>=32 bytes)}
    MINIO_ENDPOINT: http://minio:9000
    MINIO_BUCKET: eas-files
    SMTP_HOST: ${SMTP_HOST:-mailpit}
    SMTP_PORT: ${SMTP_PORT:-1025}
    TZ: Asia/Shanghai
  healthcheck:
    test: ["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]
    interval: 10s
    retries: 30
```

只有 nginx（80）和 mailpit 的 Web 界面（8025）对宿主机开放端口，数据库与 MinIO 只在 compose 内部网络可见。密码和 JWT 密钥写在 `.env` 中，`.env` 不进 git，仓库只提供 `template.env`。

Flyway 脚本：

脚本	内容
`V1__init_schema.sql`	全部表、约束、索引、序列
`V2__append_only_triggers.sql`	日志只追加触发器、模板版本不可变触发器、运行账号授权
`V3__seed_base.sql`	角色、权限点、费用类型、系统配置、部门
`V4__seed_builtin_templates.sql`	三个内置模板
`db/demo/R__seed_demo.sql`	演示账号、两个自定义模板、示例申请；仅 demo profile 加载

种子脚本可重复执行，基础数据用 `ON CONFLICT DO NOTHING`。初始化部门为研发部、市场部、财务部、总经办，演示账号统一初始密码 `Demo@2026`（以 BCrypt 哈希写入）：

用户名	姓名	部门	角色	用途
admin	系统管理员	总经办	系统管理员	管理功能；验证管理员不能审批
ceo_zhao	赵总	总经办	部门负责人、企业负责人（主办人）	高金额节点审批；企业范围查询授权
rd_head_li	李研	研发部	部门负责人	研发部审批；本人申请演示“负责人自审替代”
rd_emp_wang	王工	研发部	员工	主演示员工

用户名	姓名	部门	角色	用途
rd_emp_chen	陈工	研发部	员工	越权验证（看不到王工的申请）、同行人员
mkt_head_sun	孙经理	市场部	部门负责人	跨部门隔离验证
mkt_emp_zhou	周敏	市场部	员工	市场部员工
fin_head_qian	钱会计	财务部	部门负责人、财务审批人	财务统计授权；用章申请审批人；兜底审批人
fin_auditor_wu	吴审	财务部	财务审批人（主办人）	采购预算审查、报销财务审核
pending_he	何晓	申请加入市场部	无（待审核）	注册审核演示

示例申请让统计图表一开始就有数据：王工有已通过的差旅单（供报销关联）、审批中的采购单、已驳回的报销单各一张，周敏有一张审批中的请假单。

代码用 git 管理：`main` 为可演示版本，`dev` 为集成分支，功能在 `feature/<模块>` 分支开发；提交信息采用 `feat(flow): ...`、`fix(expense): ...` 格式；里程碑打 tag。提交作业时附带 `.git` 目录。

Q17：增强功能是如何实现的?

以下三项为【增强】，基础流程不依赖它们。

会签 / 或签：节点 `mode` 可选 `COUNTERSIGN`（会签，全部同意才通过，任一驳回即驳回）或 `OR_SIGN`（或签，任一人处理即决定节点结果，其余待办自动取消）。节点激活时为每个审批人创建待办，节点实例记录 `required_count`、`approved_count`。批准时先原子地累加计数：

FlowNodeInstanceMapper.xml

```sql
UPDATE flow_node_instance
   SET approved_count = approved_count + 1
 WHERE id = :nodeId AND status = 'ACTIVE'
RETURNING approved_count, required_count;
```

计数达到阈值（会签为全部人数，或签为 1）时才推进申请，推进仍走 Q10 中的条件更新。多人同时批准时，只有最后一个达到阈值的事务推进申请，其余事务只更新自己的待办。

催办：申请人在审批中的详情页点击“催办”，同一节点每 4 小时最多一次（`urge_record` 唯一键为节点 ID 加 4 小时时间档），当前审批人收到 `URGE` 站内通知和邮件。另有定时任务每 30 分钟扫描超过时限仍未处理的待办，每天最多提醒一次（`TASK_OVERDUE`）。

跨单重复票据预警：同一报销单内的重复票据直接拒绝（见 Q7）；跨报销单的重复只提示、不拦截，因为分次报销、发票冲红重开等合法情况也会出现同号票据，是否重复由财务判断。提交时和财务打开详情时查询：

ExpenseInvoiceMapper.xml SQL

```sql
SELECT ei.id, ei.invoice_no, a.app_no, a.status
  FROM expense_invoice ei
  JOIN application a ON a.id = ei.application_id
 WHERE ei.application_id <> :appId
   AND a.status IN ('IN_APPROVAL', 'APPROVED')
   AND (ei.file_sha256 = ANY(:hashes)
        OR ei.invoice_no_norm = ANY(:invoiceNos));
```

申请人看到黄色提示“该票据疑似已在其他报销单中使用”，仍可提交；财务审核界面用红色标签显示疑似重复的票据和对方申请编号。对方申请的正文仍受权限控制，这里只显示编号和状态。